

Progetto:

WattGuard

Titolo del documento:

Sviluppo

Document Info

Doc. Name	D2-WattGuard-sviluppo	Doc. Number	D2 V0.3
Description	Documento di sviluppo dell'applicazione		

Autori

Alessio Pesaresi	251783
Michele Porcellato	254581
Marco Piovesan	255380

Indice

1	Web API	2
2	Implementazione	77
2.1	Scelta del Tech Stack e Motivazioni Architetture	77
2.2	<i>Repository Organization</i>	78
2.3	Branching Strategy e organizzazione del lavoro	78
2.4	<i>Dependency</i>	79
2.5	<i>Database</i>	80
2.5.1	Utente	80
2.5.2	Invito	81
2.5.3	TokenResetPassword	81
2.5.4	Edificio	82
2.5.5	TipologiaEdificio	82
2.5.6	Sensore	83
2.5.7	Lecture sensori	83
2.5.8	Allerte	84
2.5.9	ConfigurazioneSistema	84
2.6	Testing	85
3	FrontEnd	93
3.1	Autenticazione	93
3.2	Accettazione Invito	94
3.3	Layout della dashboard	95
3.4	Mappa degli edifici	96
3.5	Edifici	96
3.5.1	Confronto Edifici	98
3.6	Dettaglio edificio	99
3.7	Sensori	100
3.8	Allarmi	101
3.9	Amministrazione	101
3.9.1	Utenti	102
3.9.2	Impostazioni di sistema	103
4	Deployment	104
4.1	Limitazioni del deployment	104
4.2	Configurazione CI/CD	104
4.3	Credenziali e accesso docenti	104

1 Web API

Le specifiche complete delle API sono state documentate secondo lo standard **OpenAPI 3.1.0** e sono consultabili su <https://wattguard.me/api/v1/docs> oppure su [GitHub](#)

Progettazione e note di design: L'API segue i principi *RESTful*, organizzando le risorse in modo gerarchico sotto il prefisso `/api/v1`. Si è deciso inoltre di definire gli schemi di input/output con la libreria `zod`, integrandoli nel documento OpenAPI tramite la libreria `hono-openapi`. In questo modo, si riduce il rischio che la documentazione non rispecchi il comportamento effettivo del codice. Si è inoltre deciso di utilizzare il paradigma "API Link" per permettere una navigazione più semplice e intuitiva delle risorse fornite dall'API.

```
1 openapi: 3.1.0
2 info:
3   title: WattGuard API
4   description: Energy monitoring and management API for WattGuard
5   version: 1.0.0
6 servers:
7   - url: https://wattguard.me/api/v1
8     description: WattGuard application
9 components:
10  securitySchemes:
11    bearerAuth:
12      type: http
13      scheme: bearer
14      bearerFormat: JWT
15      description: JWT token in Authorization header
16  schemas:
17    ListInvitesResponse:
18      type: object
19      properties:
20        success:
21          type: boolean
22          const: true
23        data:
24          type: array
25          items:
26            $ref: "#/components/schemas/Invite"
27          description: List of invites
28      required:
29        - success
30        - data
31    Invite:
32      type: object
33      properties:
34        self:
35          type: string
36          description: Canonical URI link to this resource
37        _id:
38          type: string
39          description: Unique invite identifier
40        email:
41          type: string
```

```

42     format: email
43     pattern: ^(?!\.)(?!.*\.\.)([A-Za-z0-9_'\+\-\.\.]*)[A-Za-z0-9_+-]@([A-Za-z0-9_
    ↪ 9][A-Za-z0-9\-\.]*\.)+[A-Za-z]{2,}$
44     description: Email address of invitee
45   role:
46     type: string
47     enum:
48       - admin
49       - operator
50     description: User role in the system
51   status:
52     type: string
53     enum:
54       - pending
55       - accepted
56       - revoked
57       - expired
58   expiresAt:
59     $ref: "#/components/schemas/IsoDateTime"
60   createdAt:
61     description: Creation timestamp
62     anyOf:
63       - $ref: "#/components/schemas/IsoDateTime"
64       - type: "null"
65   acceptedAt:
66     description: Acceptance timestamp
67     anyOf:
68       - $ref: "#/components/schemas/IsoDateTime"
69       - type: "null"
70   createdBy:
71     description: User who created the invite
72     anyOf:
73       - anyOf:
74         - type: string
75         - type: object
76           properties:
77             email:
78               type: string
79               format: email
80               pattern: ^(?!\.)(?!.*\.\.)([A-Za-z0-9_'\+\-\.\.]*)[A-Za-z0-9_+-]
    ↪ ]@([A-Za-z0-9][A-Za-z0-9\-\.]*\.)+[A-Za-z]{2,}$
81             required:
82               - email
83             - type: "null"
84   required:
85     - _id
86     - email
87     - role
88     - status
89     - expiresAt
90   IsoDateTime:
91     anyOf:
92       - type: string
93       format: date-time

```

```

94     pattern: ^(?:\d\d[2468][048]|\d\d[13579][26]|\d\d0[48]|[02468][048]00
    ↪ | [13579][26]00)-02-29|\d{4}-(?:0[13578]|1[02])-(?:0[1-9]|[12]\d|3
    ↪ [01])|(?:0[469]|11)-(?:0[1-9]|[12]\d|30)|(?:02)-(?:0[1-9]|1\d|2[0-8]
    ↪ )))T(?:0[1]\d|2[0-3]):[0-5]\d:[0-5]\d(?:\.\d+)?(?:Z))$
95     description: ISO 8601 timestamp string
96   - type: string
97     format: date-time
98   Error:
99     type: object
100    properties:
101      success:
102        type: boolean
103        const: false
104        description: Indicates failure
105      error_code:
106        type: string
107        description: Machine-readable error code
108      message:
109        type: string
110        description: Error message describing what went wrong
111      details:
112        $ref: "#/components/schemas/ErrorDetails"
113    required:
114      - success
115      - error_code
116      - message
117   ErrorDetails:
118     type: object
119     propertyName:
120       type: string
121     additionalProperties:
122       type:
123         - string
124         - number
125     description: Interpolation values for the localized error message
126   CreateInviteResponse:
127     type: object
128     properties:
129       success:
130         type: boolean
131         const: true
132       data:
133         $ref: "#/components/schemas/CreateInvite"
134     required:
135       - success
136       - data
137   CreateInvite:
138     type: object
139     properties:
140       self:
141         type: string
142         description: Canonical URI link to this resource
143       _id:
144         type: string

```

```

145     description: Unique invite identifier
146 email:
147     type: string
148     format: email
149     pattern: ^(?!\.)(?!.*\.\.)([A-Za-z0-9_'\+\-\.\.]*)[A-Za-z0-9_+-]@([A-Za-z0-9-
↵    9][A-Za-z0-9\-\.]*\.)+[A-Za-z]{2,}$
150     description: Email address of invitee
151 role:
152     type: string
153     enum:
154         - admin
155         - operator
156     description: User role in the system
157 status:
158     type: string
159     enum:
160         - pending
161         - accepted
162         - revoked
163         - expired
164 expiresAt:
165     $ref: "#/components/schemas/IsoDateTime"
166 required:
167     - _id
168     - email
169     - role
170     - status
171     - expiresAt
172 CreateInviteRequest:
173     type: object
174     properties:
175         email:
176             description: Email address
177             type: string
178             format: email
179             pattern: ^(?!\.)(?!.*\.\.)([A-Za-z0-9_'\+\-\.\.]*)[A-Za-z0-9_+-]@([A-Za-z0-9-
↵    9][A-Za-z0-9\-\.]*\.)+[A-Za-z]{2,}$
180         role:
181             type: string
182             enum:
183                 - admin
184                 - operator
185             description: User role in the system
186         required:
187             - email
188             - role
189 GetInviteByIdResponse:
190     type: object
191     properties:
192         success:
193             type: boolean
194             const: true
195         data:
196             $ref: "#/components/schemas/Invite"

```

```

197     required:
198       - success
199       - data
200   AcceptInviteResponse:
201     type: object
202     properties:
203       success:
204         type: boolean
205         const: true
206       data:
207         type: object
208         properties:
209           user:
210             $ref: "#/components/schemas/PublicUser"
211           token:
212             type: string
213             minLength: 1
214           required:
215             - user
216             - token
217     required:
218       - success
219       - data
220   PublicUser:
221     type: object
222     properties:
223       self:
224         type: string
225         description: Canonical URI link to this resource
226       _id:
227         $ref: "#/components/schemas/ObjectId"
228       email:
229         type: string
230         format: email
231         pattern: ^(?!\.)(?!.*\.\.)([A-Za-z0-9_'\+\-\.\.]*)[A-Za-z0-9_+-]@([A-Za-z0-9-
232         ↪ 9][A-Za-z0-9\-\.]*\.)+[A-Za-z]{2,}$
233         description: User email address
234       name:
235         description: User display name
236         type:
237           - string
238           - "null"
239       role:
240         type: string
241         enum:
242           - admin
243           - operator
244         description: User role in the system
245       language:
246         description: Preferred UI locale used for emails
247         anyOf:
248           - type: string
249           enum:
250             - en

```

```
250         - it
251         - de
252         description: Supported UI locale
253         - type: "null"
254     required:
255         - _id
256         - email
257         - role
258     ObjectId:
259         type: string
260         pattern: ^[0-9a-fA-F]{24}$
261         description: MongoDB ObjectId
262     AcceptInviteRequest:
263         anyOf:
264             - $ref: "#/components/schemas/AcceptInviteLocal"
265             - $ref: "#/components/schemas/AcceptInviteGoogle"
266     AcceptInviteLocal:
267         type: object
268         properties:
269             token:
270                 type: string
271                 minLength: 1
272             name:
273                 type: string
274                 minLength: 2
275                 maxLength: 64
276             password:
277                 type: string
278                 minLength: 8
279                 maxLength: 128
280         required:
281             - token
282             - name
283             - password
284     AcceptInviteGoogle:
285         type: object
286         properties:
287             token:
288                 type: string
289                 minLength: 1
290             idToken:
291                 type: string
292                 minLength: 1
293         required:
294             - token
295             - idToken
296     HealthResponse:
297         type: object
298         properties:
299             success:
300                 type: boolean
301                 const: true
302             data:
303                 type: object
```



```

304     properties:
305       status:
306         type: string
307       required:
308         - status
309     required:
310       - success
311       - data
312 SessionResponse:
313   type: object
314   properties:
315     success:
316       type: boolean
317       const: true
318     data:
319       type: object
320       properties:
321         user:
322           $ref: "#/components/schemas/PublicUser"
323         token:
324           type: string
325           minLength: 1
326       required:
327         - user
328         - token
329     required:
330       - success
331       - data
332 SessionRequest:
333   anyOf:
334     - $ref: "#/components/schemas/LocalSessionRequest"
335     - $ref: "#/components/schemas/GoogleSessionRequest"
336 LocalSessionRequest:
337   type: object
338   properties:
339     email:
340       type: string
341       format: email
342       pattern: ^((?!\\.)(?!.*\\.\\.)([A-Za-z0-9_'\+\-\.\-]*)[A-Za-z0-9_+\-]@([A-Za-z0-9\
↵    9][A-Za-z0-9\-\-]*\.)+[A-Za-z]{2,})$
343     password:
344       type: string
345       minLength: 1
346     required:
347       - email
348       - password
349 GoogleSessionRequest:
350   type: object
351   properties:
352     idToken:
353       type: string
354       minLength: 1
355     required:
356       - idToken

```

```
357 SessionUserResponse:
358   type: object
359   properties:
360     success:
361       type: boolean
362       const: true
363     data:
364       $ref: "#/components/schemas/User"
365   required:
366     - success
367     - data
368 User:
369   type: object
370   properties:
371     self:
372       type: string
373       description: Canonical URI link to this resource
374     _id:
375       $ref: "#/components/schemas/ObjectId"
376     email:
377       type: string
378       format: email
379       pattern: ^(?!\.)(?!.*\.\.)([A-Za-z0-9_'\+\-\.\.]*)([A-Za-z0-9_+-]@([A-Za-z0-9-
380         ↪ 9][A-Za-z0-9\-\.]*\.)+[A-Za-z]{2,})$
381       description: User email address
382     name:
383       description: User display name
384       type:
385         - string
386         - "null"
387     role:
388       type: string
389       enum:
390         - admin
391         - operator
392       description: User role in the system
393     isDisabled:
394       description: Whether the user account is disabled
395       type: boolean
396     language:
397       description: Preferred UI locale used for emails
398       anyOf:
399         - type: string
400           enum:
401             - en
402             - it
403             - de
404           description: Supported UI locale
405         - type: "null"
406     lastLoginAt:
407       description: Timestamp of last login
408       anyOf:
409         - $ref: "#/components/schemas/IsoDateTime"
410         - type: "null"
```

```
410     createdAt:
411       $ref: "#/components/schemas/IsoDateTime"
412   required:
413     - _id
414     - email
415     - role
416   UpdateSessionRequest:
417     type: object
418     properties:
419       language:
420         description: User preferred language for email alerts and UI
421         type: string
422         enum:
423           - en
424           - it
425           - de
426       name:
427         description: User display name
428         type: string
429         minLength: 2
430         maxLength: 64
431   GoogleConfigResponse:
432     type: object
433     properties:
434       success:
435         type: boolean
436         const: true
437       data:
438         type: object
439         properties:
440           clientId:
441             type: string
442             description: Configured Google OAuth Client ID
443           required:
444             - clientId
445       required:
446         - success
447         - data
448   CreateRecoveryTokenResponse:
449     type: object
450     properties:
451       success:
452         type: boolean
453         const: true
454       data:
455         type: object
456         properties:
457           message:
458             type: string
459             description: Success message
460           required:
461             - message
462       required:
463         - success
```

```
464     - data
465 CreateRecoveryTokenRequest:
466   type: object
467   properties:
468     email:
469       type: string
470       format: email
471       pattern: ^(?!\.)(?!.*\.\.)([A-Za-z0-9_'\+\-\.\.]*)[A-Za-z0-9_+-]@([A-Za-z0-9-
↵ 9][A-Za-z0-9\-\.]*\.)+[A-Za-z]{2,}$
472   required:
473     - email
474 ValidateRecoveryTokenResponse:
475   type: object
476   properties:
477     success:
478       type: boolean
479       const: true
480     data:
481       type: object
482       properties:
483         valid:
484           type: boolean
485           const: true
486     required:
487       - valid
488   required:
489     - success
490     - data
491 ValidateRecoveryTokenRequest:
492   type: object
493   properties:
494     token:
495       type: string
496       minLength: 1
497   required:
498     - token
499 ConfirmRecoveryResponse:
500   type: object
501   properties:
502     success:
503       type: boolean
504       const: true
505     data:
506       type: object
507       properties:
508         message:
509           type: string
510           description: Success message
511     required:
512       - message
513   required:
514     - success
515     - data
516 ConfirmRecoveryRequest:
```

```
517     type: object
518     properties:
519       token:
520         type: string
521         minLength: 1
522       password:
523         type: string
524         minLength: 8
525         maxLength: 128
526     required:
527       - token
528       - password
529   ListUsersResponse:
530     type: object
531     properties:
532       success:
533         type: boolean
534         const: true
535       data:
536         type: array
537         items:
538           $ref: "#/components/schemas/User"
539       description: List of users
540     required:
541       - success
542       - data
543   UserResponse:
544     type: object
545     properties:
546       success:
547         type: boolean
548         const: true
549       data:
550         $ref: "#/components/schemas/User"
551     required:
552       - success
553       - data
554   UpdateUserRequest:
555     type: object
556     properties:
557       role:
558         type: string
559         enum:
560           - admin
561           - operator
562       description: User role in the system
563       isDisabled:
564         description: Whether the user account is disabled
565         type: boolean
566   ListBuildingTypesResponse:
567     type: object
568     properties:
569       success:
570         type: boolean
```

```
571     const: true
572   data:
573     type: array
574     items:
575       $ref: "#/components/schemas/BuildingType"
576     description: List of building types
577   required:
578     - success
579     - data
580 BuildingType:
581   type: object
582   properties:
583     self:
584       type: string
585       description: Canonical URI link to this resource
586     _id:
587       $ref: "#/components/schemas/ObjectId"
588     name:
589       type: string
590       description: Building type name
591     description:
592       description: Optional description of the building type
593       type:
594         - string
595         - "null"
596     createdAt:
597       $ref: "#/components/schemas/IsoDateTime"
598     updatedAt:
599       $ref: "#/components/schemas/IsoDateTime"
600   required:
601     - _id
602     - name
603 BuildingTypeResponse:
604   type: object
605   properties:
606     success:
607       type: boolean
608       const: true
609     data:
610       $ref: "#/components/schemas/BuildingType"
611   required:
612     - success
613     - data
614 CreateBuildingTypeRequest:
615   type: object
616   properties:
617     name:
618       type: string
619       minLength: 1
620       description: Building type name
621     description:
622       description: Optional description
623       type: string
624   required:
```

```
625     - name
626 UpdateBuildingTypeRequest:
627   type: object
628   properties:
629     name:
630       type: string
631       minLength: 1
632       description: Building type name
633     description:
634       description: Optional description
635       type: string
636 SearchBuildingsResponse:
637   type: object
638   properties:
639     success:
640       type: boolean
641       const: true
642     data:
643       type: object
644       properties:
645         buildings:
646           type: array
647           items:
648             $ref: "#/components/schemas/BuildingSummary"
649             description: List of buildings matching search criteria
650         pagination:
651             $ref: "#/components/schemas/Pagination"
652         required:
653           - buildings
654           - pagination
655     required:
656       - success
657       - data
658 BuildingSummary:
659   type: object
660   properties:
661     self:
662       type: string
663       description: Canonical URI link to this resource
664     _id:
665       $ref: "#/components/schemas/ObjectId"
666     name:
667       type: string
668       description: Building name
669     address:
670       type: string
671       description: Building address
672     surface:
673       type: number
674       description: Surface area in square meters
675     ceilingHeight:
676       type: number
677       description: Ceiling height in meters
678     location:
```

```
679     $ref: "#/components/schemas/GeoJSONPoint"
680   buildingType:
681     anyOf:
682       - $ref: "#/components/schemas/ObjectId"
683       - $ref: "#/components/schemas/BuildingType"
684     description: Building type (ID or populated object)
685   heatingSystemType:
686     type: string
687     description: Type of heating system
688   status:
689     type: string
690     enum:
691       - active
692       - inactive
693       - decommissioned
694     description: Building operational status
695   geographicZone:
696     type: string
697     description: Geographic zone
698   activeSensors:
699     default: 0
700     description: Count of active sensors
701     type: number
702   currentConsumption:
703     default: null
704     description: Latest energy meter reading in kWh, or null if unavailable
705     type:
706       - number
707       - "null"
708   updatedAt:
709     $ref: "#/components/schemas/IsoDateTime"
710   required:
711     - _id
712     - name
713     - address
714     - surface
715     - ceilingHeight
716     - location
717     - buildingType
718     - heatingSystemType
719     - status
720     - geographicZone
721   GeoJSONPoint:
722     type: object
723   properties:
724     type:
725       type: string
726       const: Point
727       description: GeoJSON type
728   coordinates:
729     type: array
730     prefixItems:
731       - type: number
732       minimum: -180
```



```
733         maximum: 180
734         description: Longitude
735     - type: number
736         minimum: -90
737         maximum: 90
738         description: Latitude
739     items: false
740     minItems: 2
741     maxItems: 2
742     description: GeoJSON coordinates [longitude, latitude]
743     required:
744     - type
745     - coordinates
746     Pagination:
747     type: object
748     properties:
749     limit:
750     type: number
751     offset:
752     type: number
753     total:
754     type: number
755     required:
756     - limit
757     - offset
758     - total
759     description: Pagination information
760     BuildingResponse:
761     type: object
762     properties:
763     success:
764     type: boolean
765     const: true
766     data:
767     $ref: "#/components/schemas/BuildingDetail"
768     required:
769     - success
770     - data
771     BuildingDetail:
772     type: object
773     properties:
774     self:
775     type: string
776     description: Canonical URI link to this resource
777     _id:
778     $ref: "#/components/schemas/ObjectId"
779     name:
780     type: string
781     description: Building name
782     address:
783     type: string
784     description: Building address
785     surface:
786     type: number
```

```
787     description: Surface area in square meters
788 ceilingHeight:
789     type: number
790     description: Ceiling height in meters
791 location:
792     $ref: "#/components/schemas/GeoJSONPoint"
793 buildingType:
794     anyOf:
795     - $ref: "#/components/schemas/ObjectId"
796     - $ref: "#/components/schemas/BuildingType"
797     description: Building type (ID or populated object)
798 heatingSystemType:
799     type: string
800     description: Type of heating system
801 status:
802     type: string
803     enum:
804     - active
805     - inactive
806     - decommissioned
807     description: Building operational status
808 geographicZone:
809     type: string
810     description: Geographic zone
811 activeSensors:
812     default: 0
813     description: Count of active sensors
814     type: number
815 currentConsumption:
816     default: null
817     description: Latest energy meter reading in kWh, or null if unavailable
818     type:
819     - number
820     - "null"
821 updatedAt:
822     $ref: "#/components/schemas/IsoDateTime"
823 constructionYear:
824     description: Year of construction
825     anyOf:
826     - type: integer
827       minimum: -9007199254740991
828       maximum: 9007199254740991
829     - type: "null"
830 efficiencyThresholds:
831     $ref: "#/components/schemas/EfficiencyThresholds"
832 createdBy:
833     $ref: "#/components/schemas/ObjectId"
834 updatedBy:
835     $ref: "#/components/schemas/ObjectId"
836 createdAt:
837     $ref: "#/components/schemas/IsoDateTime"
838 required:
839     - _id
840     - name
```

```
841     - address
842     - surface
843     - ceilingHeight
844     - location
845     - buildingType
846     - heatingSystemType
847     - status
848     - geographicZone
849     - efficiencyThresholds
850 EfficiencyThresholds:
851   type: object
852   properties:
853     enabled:
854       type: boolean
855       description: Enable automatic efficiency alerts
856     minCop:
857       anyOf:
858         - type: number
859           minimum: 0
860           maximum: 10
861         - type: "null"
862       description: Minimum average COP; alert when below
863     required:
864       - enabled
865       - minCop
866 CreateBuildingRequest:
867   type: object
868   properties:
869     name:
870       type: string
871       minLength: 1
872       description: Building name
873     address:
874       type: string
875       minLength: 1
876       description: Building address
877     surface:
878       type: number
879       minimum: 1
880       description: Surface area in square meters
881     ceilingHeight:
882       default: 3
883       description: Ceiling height in meters
884       type: number
885       minimum: 0.5
886       maximum: 20
887     location:
888       $ref: "#/components/schemas/GeoJSONPoint"
889     buildingType:
890       description: Building type identifier or URI
891       type: string
892     heatingSystemType:
893       type: string
894       description: Type of heating system
```

```
895     constructionYear:
896       description: Year of construction
897       type: number
898       minimum: 1000
899       maximum: 2036
900     geographicZone:
901       type: string
902       minLength: 1
903       description: Geographic zone
904     efficiencyThresholds:
905       $ref: "#/components/schemas/EfficiencyThresholds"
906   required:
907     - name
908     - address
909     - surface
910     - location
911     - buildingType
912     - heatingSystemType
913     - geographicZone
914   UpdateBuildingRequest:
915     type: object
916     properties:
917       name:
918         type: string
919         minLength: 1
920         description: Building name
921       address:
922         type: string
923         minLength: 1
924         description: Building address
925       surface:
926         type: number
927         minimum: 1
928         description: Surface area in square meters
929       ceilingHeight:
930         description: Ceiling height in meters
931         type: number
932         minimum: 0.5
933         maximum: 20
934       location:
935         $ref: "#/components/schemas/GeoJSONPoint"
936       buildingType:
937         description: Building type identifier or URI
938         type: string
939       heatingSystemType:
940         type: string
941         description: Type of heating system
942     constructionYear:
943       description: Year of construction
944       type: number
945       minimum: 1000
946       maximum: 2036
947     geographicZone:
948       type: string
```

```

949     minLength: 1
950     description: Geographic zone
951 efficiencyThresholds:
952     $ref: "#/components/schemas/EfficiencyThresholds"
953 status:
954     description: Building status
955     type: string
956     enum:
957         - active
958         - inactive
959         - decommissioned
960 GetBuildingHistoryResponse:
961     type: object
962     properties:
963         success:
964             type: boolean
965             const: true
966         data:
967             $ref: "#/components/schemas/HistoricalDataResponse"
968     required:
969         - success
970         - data
971 HistoricalDataResponse:
972     type: object
973     properties:
974         buildingId:
975             $ref: "#/components/schemas/ObjectId"
976         buildingName:
977             type: string
978         period:
979             $ref: "#/components/schemas/Period"
980         data:
981             type: array
982             items:
983                 $ref: "#/components/schemas/HistoricalDataPoint"
984         description: Historical data points for graphing
985     required:
986         - buildingId
987         - buildingName
988         - period
989         - data
990 Period:
991     type: object
992     properties:
993         startDate:
994             type: string
995             format: date-time
996         pattern: ^((?:\d\d[2468] [048] |\d\d[13579] [26] |\d\d0[48] | [02468] [048] 00
997             ↪ | [13579] [26] 00)-02-29|\d{4}-(?: (?:0[13578] | 1[02])-(?:0[1-9] | [12]\d|3
998             ↪ [01])|(?:0[469] | 11)-(?:0[1-9] | [12]\d|30)|(?:02)-(?:0[1-9] | 1\d|2[0-8]
999             ↪ )))T(?: (?:[01]\d|2[0-3]):[0-5]\d:[0-5]\d(?:\.\d+)?(?:Z))$
1000     endDate:
1001         type: string
1002         format: date-time

```

```

1000     pattern: ^(?:?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48]| [02468] [048]00
    ↪ | [13579] [26]00)-02-29|\d{4}-(?:?:0[13578]|1[02])-(?:0[1-9]| [12]\d|3
    ↪ [01])|(?:0[469]|11)-(?:0[1-9]| [12]\d|30)|(?:02)-(?:0[1-9]|1\d|2[0-8]
    ↪ )))T(?:?: [01]\d|2[0-3]):[0-5]\d:[0-5]\d(?:?:\.\d+)?(?:Z))$
1001   required:
1002     - startDate
1003     - endDate
1004   HistoricalDataPoint:
1005     type: object
1006     properties:
1007       timestamp:
1008         type: string
1009         format: date-time
1010         pattern: ^(?:?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48]| [02468] [048]00
    ↪ | [13579] [26]00)-02-29|\d{4}-(?:?:0[13578]|1[02])-(?:0[1-9]| [12]\d|3
    ↪ [01])|(?:0[469]|11)-(?:0[1-9]| [12]\d|30)|(?:02)-(?:0[1-9]|1\d|2[0-8]
    ↪ )))T(?:?: [01]\d|2[0-3]):[0-5]\d:[0-5]\d(?:?:\.\d+)?(?:Z))$
1011     value:
1012       type: number
1013     unit:
1014       type: string
1015     sensorType:
1016       type: string
1017       enum:
1018         - internal_temp
1019         - external_temp
1020         - energy_meter
1021         - gas_meter
1022     description: Type of sensor (internal temperature, external temperature,
    ↪ energy
1023       meter, or gas meter)
1024     sensorId:
1025       type: string
1026   required:
1027     - timestamp
1028     - value
1029     - unit
1030     - sensorType
1031   GetBuildingEfficiencyResponse:
1032     type: object
1033     properties:
1034       success:
1035         type: boolean
1036         const: true
1037       data:
1038         $ref: "#/components/schemas/BuildingEfficiencyData"
1039     required:
1040       - success
1041       - data
1042   BuildingEfficiencyData:
1043     type: object
1044     properties:
1045       buildingId:
1046         $ref: "#/components/schemas/ObjectId"

```

```
1047     buildingName:
1048       type: string
1049     period:
1050       $ref: "#/components/schemas/Period"
1051     metrics:
1052       type: object
1053       properties:
1054         totalEnergyConsumed:
1055           type: number
1056           description: Total energy consumed in kWh
1057         averageExternalTemperature:
1058           description: Average external temperature during period
1059           type:
1060             - number
1061             - "null"
1062         estimatedHeatLossCoefficient:
1063           description: Estimated Heat Loss Coefficient (W/K)
1064           type:
1065             - number
1066             - "null"
1067         insulationQuality:
1068           description: Insulation Quality (W/(m2·K))
1069           type:
1070             - number
1071             - "null"
1072         averageCop:
1073           description: Average Coefficient of Performance (COP)
1074           type:
1075             - number
1076             - "null"
1077     required:
1078       - totalEnergyConsumed
1079       - averageExternalTemperature
1080       - estimatedHeatLossCoefficient
1081       - insulationQuality
1082       - averageCop
1083   required:
1084     - buildingId
1085     - buildingName
1086     - period
1087     - metrics
1088 ListSensorsResponse:
1089   type: object
1090   properties:
1091     success:
1092       type: boolean
1093       const: true
1094     data:
1095       type: object
1096       properties:
1097         sensors:
1098           type: array
1099           items:
1100             $ref: "#/components/schemas/Sensor"
```

```
1101         description: List of sensors
1102     pagination:
1103         $ref: "#/components/schemas/Pagination"
1104     required:
1105         - sensors
1106         - pagination
1107     required:
1108         - success
1109         - data
1110     Sensor:
1111         type: object
1112     properties:
1113         self:
1114             type: string
1115             description: Canonical URI link to this resource
1116         _id:
1117             $ref: "#/components/schemas/ObjectId"
1118         building:
1119             anyOf:
1120                 - $ref: "#/components/schemas/ObjectId"
1121                 - $ref: "#/components/schemas/PopulatedBuilding"
1122             description: Building reference (ObjectId or populated object)
1123         sensorType:
1124             type: string
1125             enum:
1126                 - internal_temp
1127                 - external_temp
1128                 - energy_meter
1129                 - gas_meter
1130             description: Type of sensor (internal temperature, external temperature,
1131                 ↪ energy
1132                 meter, or gas meter)
1133         location:
1134             type: string
1135             description: Physical location of the sensor in the building
1136         serialNumber:
1137             description: Optional sensor serial number
1138             type:
1139                 - string
1140                 - "null"
1141         installationDate:
1142             $ref: "#/components/schemas/IsoDateTime"
1143         status:
1144             type: string
1145             enum:
1146                 - active
1147                 - inactive
1148                 - maintenance
1149                 - error
1150             description: Sensor operational status
1151         lastReading:
1152             $ref: "#/components/schemas/LastReading"
1153         transmissionInterval:
1154             type: number
```



```
1154     description: "Data transmission interval in seconds (default: 90)"
1155   minThreshold:
1156     description: Minimum threshold for alerts
1157     type:
1158       - number
1159       - "null"
1160   maxThreshold:
1161     description: Maximum threshold for alerts
1162     type:
1163       - number
1164       - "null"
1165   createdBy:
1166     $ref: "#/components/schemas/ObjectId"
1167   updatedBy:
1168     $ref: "#/components/schemas/ObjectId"
1169   createdAt:
1170     $ref: "#/components/schemas/IsoDateTime"
1171   updatedAt:
1172     $ref: "#/components/schemas/IsoDateTime"
1173   required:
1174     - _id
1175     - building
1176     - sensorType
1177     - location
1178     - installationDate
1179     - status
1180     - transmissionInterval
1181   PopulatedBuilding:
1182     type: object
1183     properties:
1184       self:
1185         type: string
1186         description: Canonical URI link to this resource
1187       _id:
1188         $ref: "#/components/schemas/ObjectId"
1189       name:
1190         type: string
1191         description: Building name
1192       address:
1193         type: string
1194         description: Building address
1195     required:
1196       - _id
1197       - name
1198       - address
1199   LastReading:
1200     type: object
1201     properties:
1202       value:
1203         type: number
1204         description: Reading value
1205       timestamp:
1206         $ref: "#/components/schemas/IsoDateTime"
1207     unit:
```

```
1208     type: string
1209     description: Unit of measurement (e.g., °C, W, kWh)
1210   required:
1211     - value
1212     - timestamp
1213     - unit
1214   SensorResponse:
1215     type: object
1216     properties:
1217       success:
1218         type: boolean
1219         const: true
1220       data:
1221         $ref: "#/components/schemas/Sensor"
1222     required:
1223       - success
1224       - data
1225   CreateSensorInput:
1226     type: object
1227     properties:
1228       building:
1229         description: Building identifier or URI
1230         type: string
1231       sensorType:
1232         type: string
1233         enum:
1234           - internal_temp
1235           - external_temp
1236           - energy_meter
1237           - gas_meter
1238       description: Type of sensor (internal temperature, external temperature,
1239         ↪ energy
1240         meter, or gas meter)
1241       location:
1242         type: string
1243         minLength: 1
1244         description: Physical location in the building
1245       serialNumber:
1246         description: Optional sensor serial number
1247         type: string
1248       installationDate:
1249         $ref: "#/components/schemas/IsoDateTime"
1250       transmissionInterval:
1251         description: "Transmission interval in seconds (default: 90)"
1252         type: number
1253         minimum: 10
1254         maximum: 3600
1255       minThreshold:
1256         description: Minimum threshold for alerts
1257         type: number
1258       maxThreshold:
1259         description: Maximum threshold for alerts
1260         type: number
1261     required:
```

```
1261     - building
1262     - sensorType
1263     - location
1264 UpdateSensorRequest:
1265   type: object
1266   properties:
1267     sensorType:
1268       type: string
1269       enum:
1270         - internal_temp
1271         - external_temp
1272         - energy_meter
1273         - gas_meter
1274       description: Type of sensor (internal temperature, external temperature,
1275         ↪ energy
1276         meter, or gas meter)
1277     location:
1278       type: string
1279       minLength: 1
1280       description: Physical location in the building
1281     serialNumber:
1282       description: Optional sensor serial number
1283       type: string
1284     transmissionInterval:
1285       description: "Transmission interval in seconds (default: 90)"
1286       type: number
1287       minimum: 10
1288       maximum: 3600
1289     minThreshold:
1290       description: Minimum threshold for alerts
1291       type:
1292         - number
1293         - "null"
1294     maxThreshold:
1295       description: Maximum threshold for alerts
1296       type:
1297         - number
1298         - "null"
1299     status:
1300       type: string
1301       enum:
1302         - active
1303         - inactive
1304         - maintenance
1305         - error
1306       description: Sensor operational status
1307 GetSensorReadingsResponse:
1308   type: object
1309   properties:
1310     success:
1311       type: boolean
1312       const: true
1313     data:
1314       type: object
```

```
1314     properties:
1315       readings:
1316         type: array
1317         items:
1318           $ref: "#/components/schemas/SensorReading"
1319         description: List of sensor readings
1320       pagination:
1321         $ref: "#/components/schemas/Pagination"
1322     required:
1323       - readings
1324       - pagination
1325   required:
1326     - success
1327     - data
1328   SensorReading:
1329     type: object
1330     properties:
1331       self:
1332         type: string
1333         description: Canonical URI link to this resource
1334       _id:
1335         $ref: "#/components/schemas/ObjectId"
1336       timestamp:
1337         $ref: "#/components/schemas/IsoDateTime"
1338       value:
1339         type: number
1340         description: Reading value
1341       unit:
1342         type: string
1343         description: Unit of measurement
1344       metadata:
1345         type: object
1346         properties:
1347           sensor:
1348             $ref: "#/components/schemas/ObjectId"
1349           building:
1350             $ref: "#/components/schemas/ObjectId"
1351           sensorType:
1352             type: string
1353             enum:
1354               - internal_temp
1355               - external_temp
1356               - energy_meter
1357               - gas_meter
1358             description: Type of sensor (internal temperature, external
1359               ↪ temperature, energy
1360               meter, or gas meter)
1361           required:
1362             - sensor
1363             - sensorType
1364       required:
1365         - timestamp
1366         - value
1367         - unit
```

```
1367 ListAlertsResponse:
1368   type: object
1369   properties:
1370     success:
1371       type: boolean
1372       const: true
1373     data:
1374       type: object
1375       properties:
1376         alerts:
1377           type: array
1378           items:
1379             $ref: "#/components/schemas/Alert"
1380             description: List of alerts
1381         pagination:
1382             $ref: "#/components/schemas/Pagination"
1383       required:
1384         - alerts
1385         - pagination
1386   required:
1387     - success
1388     - data
1389 Alert:
1390   type: object
1391   properties:
1392     self:
1393       type: string
1394       description: Canonical URI link to this resource
1395     id:
1396       $ref: "#/components/schemas/ObjectId"
1397     building:
1398       anyOf:
1399         - $ref: "#/components/schemas/ObjectId"
1400         - $ref: "#/components/schemas/PopulatedBuilding"
1401       description: Building reference (ObjectId or populated object)
1402     sensor:
1403       description: Sensor reference (ObjectId or populated object)
1404       anyOf:
1405         - $ref: "#/components/schemas/ObjectId"
1406         - $ref: "#/components/schemas/PopulatedAlertSensor"
1407     type:
1408       type: string
1409       enum:
1410         - threshold_exceeded
1411         - efficiency_below_threshold
1412       description: Type of the alert
1413     thresholdType:
1414       description: Threshold direction for threshold alerts
1415       anyOf:
1416         - type: string
1417           enum:
1418             - min
1419             - max
1420         - type: "null"
```

```
1421     severity:
1422         type: string
1423         enum:
1424             - low
1425             - medium
1426             - high
1427             - critical
1428         description: Severity level
1429     value:
1430         description: Measured value that triggered the alert
1431         type:
1432             - number
1433             - "null"
1434     unit:
1435         description: Unit of the measured value
1436         type:
1437             - string
1438             - "null"
1439     limit:
1440         description: Threshold limit that was exceeded
1441         type:
1442             - number
1443             - "null"
1444     status:
1445         type: string
1446         enum:
1447             - active
1448             - acknowledged
1449             - resolved
1450         description: Current status
1451     acknowledgedBy:
1452         description: Name of the user who acknowledged
1453         type:
1454             - string
1455             - "null"
1456     acknowledgedAt:
1457         description: When it was acknowledged
1458         anyOf:
1459             - $ref: "#/components/schemas/IsoDateTime"
1460             - type: "null"
1461     resolvedBy:
1462         description: Name of the user who resolved
1463         type:
1464             - string
1465             - "null"
1466     resolvedAt:
1467         description: When it was resolved
1468         anyOf:
1469             - $ref: "#/components/schemas/IsoDateTime"
1470             - type: "null"
1471     createdAt:
1472         $ref: "#/components/schemas/IsoDateTime"
1473     updatedAt:
1474         $ref: "#/components/schemas/IsoDateTime"
```

```
1475     required:
1476       - id
1477       - building
1478       - type
1479       - severity
1480       - status
1481       - createdAt
1482       - updatedAt
1483 PopulatedAlertSensor:
1484   type: object
1485   properties:
1486     self:
1487       type: string
1488       description: Canonical URI link to this resource
1489     _id:
1490       $ref: "#/components/schemas/ObjectId"
1491     sensorType:
1492       type: string
1493       enum:
1494         - internal_temp
1495         - external_temp
1496         - energy_meter
1497         - gas_meter
1498       description: Type of the sensor
1499     location:
1500       type: string
1501       description: Physical location of the sensor in the building
1502   required:
1503     - _id
1504     - sensorType
1505     - location
1506 AlertResponse:
1507   type: object
1508   properties:
1509     success:
1510       type: boolean
1511       const: true
1512     data:
1513       $ref: "#/components/schemas/Alert"
1514   required:
1515     - success
1516     - data
1517 UpdateAlertStatusRequest:
1518   type: object
1519   properties:
1520     status:
1521       type: string
1522       enum:
1523         - acknowledged
1524         - resolved
1525       description: New status for the alert
1526   required:
1527     - status
1528 MetricsResponse:
```

```
1529     type: object
1530   properties:
1531     success:
1532       type: boolean
1533       const: true
1534     data:
1535       $ref: "#/components/schemas/SystemMetrics"
1536   required:
1537     - success
1538     - data
1539   SystemMetrics:
1540     type: object
1541     properties:
1542       sensors:
1543         type: object
1544         properties:
1545           active:
1546             type: number
1547             description: Total active sensors across all buildings
1548           total:
1549             type: number
1550             description: Total sensors across all buildings
1551         required:
1552           - active
1553           - total
1554       alerts:
1555         type: object
1556         properties:
1557           active:
1558             type: number
1559             description: Total active alerts
1560         required:
1561           - active
1562       consumption:
1563         type: object
1564         properties:
1565           electricity:
1566             description: Sum of current energy_meter readings (kWh)
1567             type:
1568               - number
1569               - "null"
1570           gas:
1571             description: Sum of current gas_meter readings (m³)
1572             type:
1573               - number
1574               - "null"
1575         required:
1576           - electricity
1577           - gas
1578     required:
1579       - sensors
1580       - alerts
1581       - consumption
1582   MetricsHistoryResponse:
```



```
1583     type: object
1584   properties:
1585     success:
1586       type: boolean
1587       const: true
1588     data:
1589       $ref: "#/components/schemas/MetricsHistoryData"
1590   required:
1591     - success
1592     - data
1593 MetricsHistoryData:
1594   type: object
1595   properties:
1596     period:
1597       type: object
1598       properties:
1599         startDate:
1600           type: string
1601         endDate:
1602           type: string
1603         interval:
1604           type: string
1605         enum:
1606           - hour
1607           - day
1608           - week
1609       required:
1610         - startDate
1611         - endDate
1612         - interval
1613     data:
1614       type: array
1615       items:
1616         $ref: "#/components/schemas/MetricsHistoryDataPoint"
1617   required:
1618     - period
1619     - data
1620 MetricsHistoryDataPoint:
1621   type: object
1622   properties:
1623     date:
1624       type: string
1625       description: Bucket label (ISO date or hour string)
1626     electricity:
1627       description: Average energy_meter reading for the bucket (kWh)
1628       type:
1629         - number
1630         - "null"
1631     gas:
1632       description: Average gas_meter reading for the bucket (m³)
1633       type:
1634         - number
1635         - "null"
1636   required:
```

```
1637     - date
1638     - electricity
1639     - gas
1640 SettingsResponse:
1641   type: object
1642   properties:
1643     success:
1644       type: boolean
1645       const: true
1646     data:
1647       $ref: "#/components/schemas/SystemConfig"
1648   required:
1649     - success
1650     - data
1651 SystemConfig:
1652   type: object
1653   properties:
1654     polling:
1655       $ref: "#/components/schemas/PollingConfig"
1656     notifications:
1657       $ref: "#/components/schemas/NotificationsConfig"
1658     database:
1659       $ref: "#/components/schemas/DatabaseConfig"
1660   required:
1661     - polling
1662     - notifications
1663     - database
1664 PollingConfig:
1665   type: object
1666   properties:
1667     intervalSeconds:
1668       type: integer
1669       minimum: 10
1670       maximum: 3600
1671       description: Frontend refetch interval in seconds for the realtime widget
1672     autoPollingEnabled:
1673       type: boolean
1674       description: Whether automatic polling is enabled
1675   required:
1676     - intervalSeconds
1677     - autoPollingEnabled
1678 NotificationsConfig:
1679   type: object
1680   properties:
1681     emailEnabled:
1682       type: boolean
1683       description: Whether email notifications are enabled
1684   required:
1685     - emailEnabled
1686 DatabaseConfig:
1687   type: object
1688   properties:
1689     dataRetentionDays:
1690       type: integer
```

```
1691         minimum: 1
1692         maximum: 9007199254740991
1693         description: Number of days to retain historical sensor readings
1694     required:
1695     - dataRetentionDays
1696 UpdateSettingsRequest:
1697     type: object
1698     properties:
1699     polling:
1700     type: object
1701     properties:
1702     intervalSeconds:
1703     type: integer
1704     minimum: 10
1705     maximum: 3600
1706     description: Frontend refetch interval in seconds for the realtime
1707     ↪ widget
1708     autoPollingEnabled:
1709     type: boolean
1710     description: Whether automatic polling is enabled
1711     notifications:
1712     type: object
1713     properties:
1714     emailEnabled:
1715     type: boolean
1716     description: Whether email notifications are enabled
1717     database:
1718     type: object
1719     properties:
1720     dataRetentionDays:
1721     type: integer
1722     minimum: 1
1723     maximum: 9007199254740991
1724     description: Number of days to retain historical sensor readings
1725 tags:
1726 - name: Authentication
1727   description: User authentication, session lifecycle, Google OAuth
1728   ↪ configuration,
1729   and password recovery endpoints
1730 - name: Users
1731   description: User management endpoints (admin only)
1732 - name: Invites
1733   description: User invitation management
1734 - name: Health
1735   description: API health check endpoints
1736 - name: Building Types
1737   description: Building type management - CRUD operations for configurable
1738   building types (school, office, library, etc.)
1739 - name: Buildings
1740   description: Building management
1741 - name: Sensors
1742   description: Sensor management
1743 - name: Alerts
1744   description: Energy efficiency and sensor alerts management
```

```
1743 - name: Metrics
1744   description: Building metrics and baseline statistics
1745 - name: Readings
1746   description: Sensor readings ingestion and historical queries
1747 - name: Reports
1748   description: Building energy report generation and exports
1749 - name: Settings
1750   description: System and dashboard settings configuration
1751 - name: Backups
1752   description: Database backup and restore operations
1753 paths:
1754   /invites:
1755     get:
1756       operationId: getInvites
1757       summary: List invites or verify by token
1758       description: "Without query: lists all invites (admin only). With ?token=:
1759         public lookup of a single invite for registration"
1760       tags:
1761         - Invites
1762       responses:
1763         "200":
1764           description: List of invites, or single invite lookup by token
1765           content:
1766             application/json:
1767               schema:
1768                 $ref: "#/components/schemas/ListInvitesResponse"
1769         "400":
1770           description: Validation error
1771           content:
1772             application/json:
1773               schema:
1774                 $ref: "#/components/schemas/Error"
1775         "401":
1776           description: Unauthorized - Invalid or missing JWT token
1777           content:
1778             application/json:
1779               schema:
1780                 $ref: "#/components/schemas/Error"
1781         "403":
1782           description: Forbidden - Requires admin role
1783           content:
1784             application/json:
1785               schema:
1786                 $ref: "#/components/schemas/Error"
1787         "404":
1788           description: Invite token not found
1789           content:
1790             application/json:
1791               schema:
1792                 $ref: "#/components/schemas/Error"
1793         "422":
1794           description: Invite is expired or already used
1795           content:
1796             application/json:
```

```
1797         schema:
1798             $ref: "#/components/schemas/Error"
1799     parameters:
1800     - in: query
1801       name: token
1802       schema:
1803         type: string
1804         minLength: 1
1805       description: Invitation token for public lookup
1806     post:
1807       operationId: postInvites
1808       summary: Create invite
1809       description: Creates an invite and sends the registration email (admin only,
1810         valid for 7 days)
1811       tags:
1812       - Invites
1813       security:
1814       - bearerAuth: []
1815       responses:
1816         "201":
1817           description: Invite created and email sent successfully
1818           content:
1819             application/json:
1820               schema:
1821                 $ref: "#/components/schemas/CreateInviteResponse"
1822         "400":
1823           description: Invalid request - validation error
1824           content:
1825             application/json:
1826               schema:
1827                 $ref: "#/components/schemas/Error"
1828         "401":
1829           description: Unauthorized - Invalid or missing JWT token
1830           content:
1831             application/json:
1832               schema:
1833                 $ref: "#/components/schemas/Error"
1834         "403":
1835           description: Forbidden - Requires admin role
1836           content:
1837             application/json:
1838               schema:
1839                 $ref: "#/components/schemas/Error"
1840         "409":
1841           description: Conflict - User already exists or pending invite exists
1842           content:
1843             application/json:
1844               schema:
1845                 $ref: "#/components/schemas/Error"
1846         "500":
1847           description: Failed to send invitation email
1848           content:
1849             application/json:
1850               schema:
```

```
1851         $ref: "#/components/schemas/Error"
1852     requestBody:
1853         required: true
1854         content:
1855             application/json:
1856                 schema:
1857                     $ref: "#/components/schemas/CreateInviteRequest"
1858 /invites/{id}:
1859     get:
1860         operationId: getInvitesById
1861         summary: Get invite by ID
1862         description: Returns full invite data via its canonical ID (admin only)
1863         tags:
1864             - Invites
1865         security:
1866             - bearerAuth: []
1867         responses:
1868             "200":
1869                 description: Invite details retrieved successfully
1870                 content:
1871                     application/json:
1872                         schema:
1873                             $ref: "#/components/schemas/GetInviteByIdResponse"
1874             "400":
1875                 description: Invalid ID parameter
1876                 content:
1877                     application/json:
1878                         schema:
1879                             $ref: "#/components/schemas/Error"
1880             "401":
1881                 description: Unauthorized - Invalid or missing JWT token
1882                 content:
1883                     application/json:
1884                         schema:
1885                             $ref: "#/components/schemas/Error"
1886             "403":
1887                 description: Forbidden - Requires admin role
1888                 content:
1889                     application/json:
1890                         schema:
1891                             $ref: "#/components/schemas/Error"
1892             "404":
1893                 description: Invite not found
1894                 content:
1895                     application/json:
1896                         schema:
1897                             $ref: "#/components/schemas/Error"
1898         parameters:
1899             - in: path
1900               name: id
1901               schema:
1902                   $ref: "#/components/schemas/ObjectId"
1903               required: true
1904     delete:
```

```
1905     operationId: deleteInvitesById
1906     summary: Revoke invite
1907     description: Revokes a pending invite (admin only)
1908     tags:
1909       - Invites
1910     security:
1911       - bearerAuth: []
1912     responses:
1913       "204":
1914         description: Invite revoked successfully
1915       "400":
1916         description: Validation error
1917         content:
1918           application/json:
1919             schema:
1920               $ref: "#/components/schemas/Error"
1921       "401":
1922         description: Unauthorized - Invalid or missing JWT token
1923         content:
1924           application/json:
1925             schema:
1926               $ref: "#/components/schemas/Error"
1927       "403":
1928         description: Forbidden - Requires admin role
1929         content:
1930           application/json:
1931             schema:
1932               $ref: "#/components/schemas/Error"
1933       "404":
1934         description: Invite not found
1935         content:
1936           application/json:
1937             schema:
1938               $ref: "#/components/schemas/Error"
1939       "409":
1940         description: Invite cannot be revoked (not pending)
1941         content:
1942           application/json:
1943             schema:
1944               $ref: "#/components/schemas/Error"
1945     parameters:
1946       - in: path
1947         name: id
1948         schema:
1949           $ref: "#/components/schemas/ObjectId"
1950         required: true
1951     patch:
1952       operationId: patchInvitesById
1953       summary: Accept invite
1954       description: Transitions pending -> accepted on the canonical URI. Token in
1955         ↪ body
1956         (never in path). Creates the account (password or Google) and opens the
1957         session
1958       tags:
```

```
1958     - Invites
1959 responses:
1960   "200":
1961     description: Invitation accepted successfully, user created and session
1962       established
1963     content:
1964       application/json:
1965         schema:
1966           $ref: "#/components/schemas/AcceptInviteResponse"
1967   "400":
1968     description: Validation error
1969     content:
1970       application/json:
1971         schema:
1972           $ref: "#/components/schemas/Error"
1973   "401":
1974     description: Invalid Google ID token
1975     content:
1976       application/json:
1977         schema:
1978           $ref: "#/components/schemas/Error"
1979   "404":
1980     description: Invite not found
1981     content:
1982       application/json:
1983         schema:
1984           $ref: "#/components/schemas/Error"
1985   "409":
1986     description: User already exists or Google account already linked to
1987       ↪ another user
1988     content:
1989       application/json:
1990         schema:
1991           $ref: "#/components/schemas/Error"
1992   "422":
1993     description: Invalid or expired invite or email mismatch
1994     content:
1995       application/json:
1996         schema:
1997           $ref: "#/components/schemas/Error"
1998   "503":
1999     description: Google OAuth not configured
2000     content:
2001       application/json:
2002         schema:
2003           $ref: "#/components/schemas/Error"
2004 parameters:
2005   - in: path
2006     name: id
2007     schema:
2008       $ref: "#/components/schemas/ObjectId"
2009     required: true
2010 requestBody:
2011   required: true
```



```
2011     content:
2012       application/json:
2013         schema:
2014           $ref: "#/components/schemas/AcceptInviteRequest"
2015 /health:
2016   get:
2017     operationId: getHealth
2018     summary: Check server status
2019     description: Checks that the server is running (public)
2020     tags:
2021       - Health
2022     responses:
2023       "200":
2024         description: Server is running correctly
2025         content:
2026           application/json:
2027             schema:
2028               $ref: "#/components/schemas/HealthResponse"
2029 /auth/session:
2030   post:
2031     operationId: postAuthSession
2032     summary: Create session
2033     description: Creates a session via email+password or Google ID token; returns
2034       ↪ a
2035       Bearer token
2036     tags:
2037       - Authentication
2038     responses:
2039       "200":
2040         description: Session created successfully
2041         content:
2042           application/json:
2043             schema:
2044               $ref: "#/components/schemas/SessionResponse"
2045       "400":
2046         description: Validation error
2047         content:
2048           application/json:
2049             schema:
2050               $ref: "#/components/schemas/Error"
2051       "401":
2052         description: Invalid credentials or invalid Google ID token
2053         content:
2054           application/json:
2055             schema:
2056               $ref: "#/components/schemas/Error"
2057       "403":
2058         description: Account is disabled
2059         content:
2060           application/json:
2061             schema:
2062               $ref: "#/components/schemas/Error"
2063       "404":
2064         description: User account not found
```

```
2064         content:
2065             application/json:
2066                 schema:
2067                     $ref: "#/components/schemas/Error"
2068 "422":
2069     description: Unverified email or account mismatch
2070     content:
2071         application/json:
2072             schema:
2073                 $ref: "#/components/schemas/Error"
2074 "503":
2075     description: Authentication provider not configured
2076     content:
2077         application/json:
2078             schema:
2079                 $ref: "#/components/schemas/Error"
2080 requestBody:
2081     required: true
2082     content:
2083         application/json:
2084             schema:
2085                 $ref: "#/components/schemas/SessionRequest"
2086 get:
2087     operationId: getAuthSession
2088     summary: Get current session
2089     description: Returns the authenticated user and session data
2090     tags:
2091         - Authentication
2092     security:
2093         - bearerAuth: []
2094     responses:
2095         "200":
2096             description: Current user and session information
2097             content:
2098                 application/json:
2099                     schema:
2100                         $ref: "#/components/schemas/SessionUserResponse"
2101         "401":
2102             description: Unauthorized - Invalid or missing JWT token
2103             content:
2104                 application/json:
2105                     schema:
2106                         $ref: "#/components/schemas/Error"
2107 patch:
2108     operationId: patchAuthSession
2109     summary: Update current profile
2110     description: Updates the authenticated user's name and preferences (e.g.
2111         ↪ language)
2112     tags:
2113         - Authentication
2114     security:
2115         - bearerAuth: []
2116     responses:
2117         "200":
```

```
2117     description: Updated user information
2118     content:
2119       application/json:
2120         schema:
2121           $ref: "#/components/schemas/SessionUserResponse"
2122 "400":
2123   description: Validation error
2124   content:
2125     application/json:
2126       schema:
2127         $ref: "#/components/schemas/Error"
2128 "401":
2129   description: Unauthorized - Invalid or missing JWT token
2130   content:
2131     application/json:
2132       schema:
2133         $ref: "#/components/schemas/Error"
2134 requestBody:
2135   required: true
2136   content:
2137     application/json:
2138       schema:
2139         $ref: "#/components/schemas/UpdateSessionRequest"
2140 /auth/google/config:
2141   get:
2142     operationId: getAuthGoogleConfig
2143     summary: Get Google OAuth config
2144     description: Returns the public Google Client ID for the frontend SDK
2145     tags:
2146       - Authentication
2147     responses:
2148       "200":
2149         description: Google configuration
2150         content:
2151           application/json:
2152             schema:
2153               $ref: "#/components/schemas/GoogleConfigResponse"
2154 /auth/recovery-tokens:
2155   post:
2156     operationId: postAuthRecoveryTokens
2157     summary: Request password recovery
2158     description: Creates a recovery token and sends the email if the account
2159       ↪ exists
2159     tags:
2160       - Authentication
2161     responses:
2162       "200":
2163         description: Request processed
2164         content:
2165           application/json:
2166             schema:
2167               $ref: "#/components/schemas/CreateRecoveryTokenResponse"
2168       "400":
2169         description: Validation error
```

```
2170         content:
2171             application/json:
2172                 schema:
2173                     $ref: "#/components/schemas/Error"
2174 requestBody:
2175     required: true
2176     content:
2177         application/json:
2178             schema:
2179                 $ref: "#/components/schemas/CreateRecoveryTokenRequest"
2180 /auth/recovery-validations:
2181     post:
2182         operationId: postAuthRecoveryValidations
2183         summary: Verify recovery token
2184         description: Checks recovery token validity without consuming it. Token in
2185             ↪ body,
2186             never in URL
2187         tags:
2188             - Authentication
2189         responses:
2190             "200":
2191                 description: Token is valid
2192                 content:
2193                     application/json:
2194                         schema:
2195                             $ref: "#/components/schemas/ValidateRecoveryTokenResponse"
2196             "400":
2197                 description: Validation error
2198                 content:
2199                     application/json:
2200                         schema:
2201                             $ref: "#/components/schemas/Error"
2202             "404":
2203                 description: Token not found
2204                 content:
2205                     application/json:
2206                         schema:
2207                             $ref: "#/components/schemas/Error"
2208             "422":
2209                 description: Token has expired
2210                 content:
2211                     application/json:
2212                         schema:
2213                             $ref: "#/components/schemas/Error"
2214         requestBody:
2215             required: true
2216             content:
2217                 application/json:
2218                     schema:
2219                         $ref: "#/components/schemas/ValidateRecoveryTokenRequest"
2220 /auth/recovery-confirmations:
2221     post:
2222         operationId: postAuthRecoveryConfirmations
2223         summary: Confirm password recovery
```

```
2223     description: Resets the password with a valid single-use token
2224     tags:
2225       - Authentication
2226     responses:
2227       "200":
2228         description: Password recovery successful
2229         content:
2230           application/json:
2231             schema:
2232               $ref: "#/components/schemas/ConfirmRecoveryResponse"
2233       "400":
2234         description: Validation error
2235         content:
2236           application/json:
2237             schema:
2238               $ref: "#/components/schemas/Error"
2239       "404":
2240         description: Token or user not found
2241         content:
2242           application/json:
2243             schema:
2244               $ref: "#/components/schemas/Error"
2245       "422":
2246         description: Token has expired
2247         content:
2248           application/json:
2249             schema:
2250               $ref: "#/components/schemas/Error"
2251     requestBody:
2252       required: true
2253       content:
2254         application/json:
2255           schema:
2256             $ref: "#/components/schemas/ConfirmRecoveryRequest"
2257 /users:
2258   get:
2259     operationId: getUsers
2260     summary: List users
2261     description: Returns all registered users (admin only)
2262     tags:
2263       - Users
2264     security:
2265       - bearerAuth: []
2266     responses:
2267       "200":
2268         description: List of users retrieved successfully
2269         content:
2270           application/json:
2271             schema:
2272               $ref: "#/components/schemas/ListUsersResponse"
2273       "401":
2274         description: Unauthorized - Invalid or missing JWT token
2275         content:
2276           application/json:
```

```
2277         schema:
2278             $ref: "#/components/schemas/Error"
2279     "403":
2280         description: Forbidden - Requires admin role
2281         content:
2282             application/json:
2283                 schema:
2284                     $ref: "#/components/schemas/Error"
2285 /users/{id}:
2286     get:
2287         operationId: getUsersById
2288         summary: Get user
2289         description: Returns a single user by ID (admin only)
2290         tags:
2291             - Users
2292         security:
2293             - bearerAuth: []
2294         responses:
2295             "200":
2296                 description: User details retrieved successfully
2297                 content:
2298                     application/json:
2299                         schema:
2300                             $ref: "#/components/schemas/UserResponse"
2301             "400":
2302                 description: Invalid user ID parameter
2303                 content:
2304                     application/json:
2305                         schema:
2306                             $ref: "#/components/schemas/Error"
2307             "401":
2308                 description: Unauthorized - Invalid or missing JWT token
2309                 content:
2310                     application/json:
2311                         schema:
2312                             $ref: "#/components/schemas/Error"
2313             "403":
2314                 description: Forbidden - Requires admin role
2315                 content:
2316                     application/json:
2317                         schema:
2318                             $ref: "#/components/schemas/Error"
2319             "404":
2320                 description: User not found
2321                 content:
2322                     application/json:
2323                         schema:
2324                             $ref: "#/components/schemas/Error"
2325         parameters:
2326             - in: path
2327               name: id
2328               schema:
2329                   $ref: "#/components/schemas/ObjectId"
2330               required: true
```

```
2331 patch:
2332   operationId: patchUsersById
2333   summary: Update user
2334   description: Updates a user role or disabled flag (admin only, not self)
2335   tags:
2336     - Users
2337   security:
2338     - bearerAuth: []
2339   responses:
2340     "200":
2341       description: User updated successfully
2342       content:
2343         application/json:
2344           schema:
2345             $ref: "#/components/schemas/UserResponse"
2346     "400":
2347       description: Validation error
2348       content:
2349         application/json:
2350           schema:
2351             $ref: "#/components/schemas/Error"
2352     "401":
2353       description: Unauthorized - Invalid or missing JWT token
2354       content:
2355         application/json:
2356           schema:
2357             $ref: "#/components/schemas/Error"
2358     "403":
2359       description: Forbidden - Requires admin role
2360       content:
2361         application/json:
2362           schema:
2363             $ref: "#/components/schemas/Error"
2364     "404":
2365       description: User not found
2366       content:
2367         application/json:
2368           schema:
2369             $ref: "#/components/schemas/Error"
2370     "422":
2371       description: Cannot update your own account or empty update body
2372       content:
2373         application/json:
2374           schema:
2375             $ref: "#/components/schemas/Error"
2376   parameters:
2377     - in: path
2378       name: id
2379       schema:
2380         $ref: "#/components/schemas/ObjectId"
2381       required: true
2382   requestBody:
2383     required: true
2384     content:
```

```
2385     application/json:
2386       schema:
2387         $ref: "#/components/schemas/UpdateUserRequest"
2388 delete:
2389   operationId: deleteUsersById
2390   summary: Delete user
2391   description: Deletes a user (admin only, not self)
2392   tags:
2393     - Users
2394   security:
2395     - bearerAuth: []
2396   responses:
2397     "204":
2398       description: User deleted successfully
2399     "400":
2400       description: Validation error
2401       content:
2402         application/json:
2403           schema:
2404             $ref: "#/components/schemas/Error"
2405     "401":
2406       description: Unauthorized - Invalid or missing JWT token
2407       content:
2408         application/json:
2409           schema:
2410             $ref: "#/components/schemas/Error"
2411     "403":
2412       description: Forbidden - Requires admin role
2413       content:
2414         application/json:
2415           schema:
2416             $ref: "#/components/schemas/Error"
2417     "404":
2418       description: User not found
2419       content:
2420         application/json:
2421           schema:
2422             $ref: "#/components/schemas/Error"
2423     "422":
2424       description: Cannot delete your own account
2425       content:
2426         application/json:
2427           schema:
2428             $ref: "#/components/schemas/Error"
2429   parameters:
2430     - in: path
2431       name: id
2432       schema:
2433         $ref: "#/components/schemas/ObjectId"
2434       required: true
2435 /building-types:
2436   get:
2437     operationId: getBuildingTypes
2438     summary: List building types
```



```
2439     description: Returns all building types ordered by name
2440     tags:
2441       - Building Types
2442     security:
2443       - bearerAuth: []
2444     responses:
2445       "200":
2446         description: Building types retrieved successfully
2447         content:
2448           application/json:
2449             schema:
2450               $ref: "#/components/schemas/ListBuildingTypesResponse"
2451       "401":
2452         description: Unauthorized - Invalid or missing JWT token
2453         content:
2454           application/json:
2455             schema:
2456               $ref: "#/components/schemas/Error"
2457       "403":
2458         description: Forbidden - Requires admin or operator role
2459         content:
2460           application/json:
2461             schema:
2462               $ref: "#/components/schemas/Error"
2463   post:
2464     operationId: postBuildingTypes
2465     summary: Create building type
2466     description: Creates a new building type (admin only, unique name)
2467     tags:
2468       - Building Types
2469     security:
2470       - bearerAuth: []
2471     responses:
2472       "201":
2473         description: Building type created successfully
2474         content:
2475           application/json:
2476             schema:
2477               $ref: "#/components/schemas/BuildingTypeResponse"
2478       "400":
2479         description: Validation error
2480         content:
2481           application/json:
2482             schema:
2483               $ref: "#/components/schemas/Error"
2484       "401":
2485         description: Unauthorized - Invalid or missing JWT token
2486         content:
2487           application/json:
2488             schema:
2489               $ref: "#/components/schemas/Error"
2490       "403":
2491         description: Forbidden - Requires admin role
2492         content:
```

```
2493         application/json:
2494             schema:
2495                 $ref: "#/components/schemas/Error"
2496     "409":
2497         description: Duplicate building type name
2498         content:
2499             application/json:
2500                 schema:
2501                     $ref: "#/components/schemas/Error"
2502     requestBody:
2503         required: true
2504         content:
2505             application/json:
2506                 schema:
2507                     $ref: "#/components/schemas/CreateBuildingTypeRequest"
2508 /building-types/{id}:
2509     get:
2510         operationId: getBuildingTypesById
2511         summary: Get building type
2512         description: Returns building type detail by ID
2513         tags:
2514             - Building Types
2515         security:
2516             - bearerAuth: []
2517         responses:
2518             "200":
2519                 description: Building type retrieved successfully
2520                 content:
2521                     application/json:
2522                         schema:
2523                             $ref: "#/components/schemas/BuildingTypeResponse"
2524             "400":
2525                 description: Invalid ID parameter
2526                 content:
2527                     application/json:
2528                         schema:
2529                             $ref: "#/components/schemas/Error"
2530             "401":
2531                 description: Unauthorized - Invalid or missing JWT token
2532                 content:
2533                     application/json:
2534                         schema:
2535                             $ref: "#/components/schemas/Error"
2536             "403":
2537                 description: Forbidden - Requires admin or operator role
2538                 content:
2539                     application/json:
2540                         schema:
2541                             $ref: "#/components/schemas/Error"
2542             "404":
2543                 description: Building type not found
2544                 content:
2545                     application/json:
2546                         schema:
```

```
2547         $ref: "#/components/schemas/Error"
2548     parameters:
2549     - in: path
2550       name: id
2551       schema:
2552         $ref: "#/components/schemas/ObjectId"
2553       required: true
2554     patch:
2555       operationId: patchBuildingTypesById
2556       summary: Update building type
2557       description: Updates a building type name or description (admin only)
2558       tags:
2559       - Building Types
2560       security:
2561       - bearerAuth: []
2562     responses:
2563       "200":
2564         description: Building type updated successfully
2565         content:
2566           application/json:
2567             schema:
2568               $ref: "#/components/schemas/BuildingTypeResponse"
2569       "400":
2570         description: Validation error
2571         content:
2572           application/json:
2573             schema:
2574               $ref: "#/components/schemas/Error"
2575       "401":
2576         description: Unauthorized - Invalid or missing JWT token
2577         content:
2578           application/json:
2579             schema:
2580               $ref: "#/components/schemas/Error"
2581       "403":
2582         description: Forbidden - Requires admin role
2583         content:
2584           application/json:
2585             schema:
2586               $ref: "#/components/schemas/Error"
2587       "404":
2588         description: Building type not found
2589         content:
2590           application/json:
2591             schema:
2592               $ref: "#/components/schemas/Error"
2593       "409":
2594         description: Building type with this name already exists
2595         content:
2596           application/json:
2597             schema:
2598               $ref: "#/components/schemas/Error"
2599     parameters:
2600     - in: path
```

```
2601     name: id
2602     schema:
2603       $ref: "#/components/schemas/ObjectId"
2604     required: true
2605 requestBody:
2606   required: true
2607   content:
2608     application/json:
2609       schema:
2610         $ref: "#/components/schemas/UpdateBuildingTypeRequest"
2611 delete:
2612   operationId: deleteBuildingTypesById
2613   summary: Delete building type
2614   description: Deletes the type if not used by buildings (admin only)
2615   tags:
2616     - Building Types
2617   security:
2618     - bearerAuth: []
2619   responses:
2620     "204":
2621       description: Building type deleted successfully
2622     "400":
2623       description: Validation error
2624       content:
2625         application/json:
2626           schema:
2627             $ref: "#/components/schemas/Error"
2628     "401":
2629       description: Unauthorized - Invalid or missing JWT token
2630       content:
2631         application/json:
2632           schema:
2633             $ref: "#/components/schemas/Error"
2634     "403":
2635       description: Forbidden - Requires admin role
2636       content:
2637         application/json:
2638           schema:
2639             $ref: "#/components/schemas/Error"
2640     "404":
2641       description: Building type not found
2642       content:
2643         application/json:
2644           schema:
2645             $ref: "#/components/schemas/Error"
2646     "409":
2647       description: Cannot delete - building type is in use
2648       content:
2649         application/json:
2650           schema:
2651             $ref: "#/components/schemas/Error"
2652 parameters:
2653   - in: path
2654     name: id
```

```
2655     schema:
2656       $ref: "#/components/schemas/ObjectId"
2657     required: true
2658 /buildings:
2659   get:
2660     operationId: getBuildings
2661     summary: Search buildings
2662     description: Searches buildings with filters, pagination and sorting
2663     tags:
2664       - Buildings
2665     security:
2666       - bearerAuth: []
2667     responses:
2668       "200":
2669         description: Buildings retrieved successfully
2670         content:
2671           application/json:
2672             schema:
2673               $ref: "#/components/schemas/SearchBuildingsResponse"
2674       "400":
2675         description: Invalid query parameters
2676         content:
2677           application/json:
2678             schema:
2679               $ref: "#/components/schemas/Error"
2680       "401":
2681         description: Unauthorized
2682         content:
2683           application/json:
2684             schema:
2685               $ref: "#/components/schemas/Error"
2686       "403":
2687         description: Forbidden
2688         content:
2689           application/json:
2690             schema:
2691               $ref: "#/components/schemas/Error"
2692     parameters:
2693       - in: query
2694         name: limit
2695         schema:
2696           type: string
2697         description: "Maximum number of results (1-100, default: 50)"
2698       - in: query
2699         name: offset
2700         schema:
2701           type: string
2702         description: "Number of results to skip (default: 0)"
2703       - in: query
2704         name: name
2705         schema:
2706           type: string
2707         description: Filter by building name (partial match)
2708       - in: query
```

```
2709     name: address
2710     schema:
2711       type: string
2712     description: Filter by address (partial match)
2713   - in: query
2714     name: zone
2715     schema:
2716       type: string
2717     description: Filter by geographic zone
2718   - in: query
2719     name: buildingType
2720     schema:
2721       $ref: "#/components/schemas/ObjectId"
2722   - in: query
2723     name: status
2724     schema:
2725       type: string
2726       enum:
2727         - active
2728         - inactive
2729         - decommissioned
2730     description: Filter by operational status
2731   - in: query
2732     name: sortBy
2733     schema:
2734       type: string
2735       enum:
2736         - name
2737         - createdAt
2738         - updatedAt
2739     description: "Sort field (default: updatedAt)"
2740   - in: query
2741     name: sortOrder
2742     schema:
2743       type: string
2744       enum:
2745         - asc
2746         - desc
2747     description: "Sort order (default: desc)"
2748 post:
2749   operationId: postBuildings
2750   summary: Create building
2751   description: Creates a new building after validating type and plant
2752   tags:
2753     - Buildings
2754   security:
2755     - bearerAuth: []
2756   responses:
2757     "201":
2758       description: Building created successfully
2759       content:
2760         application/json:
2761           schema:
2762             $ref: "#/components/schemas/BuildingResponse"
```

```
2763     "400":
2764       description: Validation error
2765       content:
2766         application/json:
2767           schema:
2768             $ref: "#/components/schemas/Error"
2769     "401":
2770       description: Unauthorized
2771       content:
2772         application/json:
2773           schema:
2774             $ref: "#/components/schemas/Error"
2775     "403":
2776       description: Forbidden
2777       content:
2778         application/json:
2779           schema:
2780             $ref: "#/components/schemas/Error"
2781     "422":
2782       description: Validation error
2783       content:
2784         application/json:
2785           schema:
2786             $ref: "#/components/schemas/Error"
2787 requestBody:
2788   required: true
2789   content:
2790     application/json:
2791       schema:
2792         $ref: "#/components/schemas/CreateBuildingRequest"
2793 /buildings/{id}:
2794   get:
2795     operationId: getBuildingsById
2796     summary: Get building
2797     description: Returns detail with active sensors and consumption
2798     tags:
2799       - Buildings
2800     security:
2801       - bearerAuth: []
2802     responses:
2803       "200":
2804         description: Building details retrieved successfully
2805         content:
2806           application/json:
2807             schema:
2808               $ref: "#/components/schemas/BuildingResponse"
2809       "400":
2810         description: Invalid ID parameter
2811         content:
2812           application/json:
2813             schema:
2814               $ref: "#/components/schemas/Error"
2815       "401":
2816         description: Unauthorized
```

```
2817         content:
2818             application/json:
2819                 schema:
2820                     $ref: "#/components/schemas/Error"
2821 "403":
2822     description: Forbidden
2823     content:
2824         application/json:
2825             schema:
2826                 $ref: "#/components/schemas/Error"
2827 "404":
2828     description: Building not found
2829     content:
2830         application/json:
2831             schema:
2832                 $ref: "#/components/schemas/Error"
2833 parameters:
2834     - in: path
2835       name: id
2836       schema:
2837           $ref: "#/components/schemas/ObjectId"
2838       required: true
2839 patch:
2840     operationId: patchBuildingsById
2841     summary: Update building
2842     description: Updates data and handles efficiency/district-heating thresholds
2843     tags:
2844         - Buildings
2845     security:
2846         - bearerAuth: []
2847     responses:
2848         "200":
2849             description: Building updated successfully
2850             content:
2851                 application/json:
2852                     schema:
2853                         $ref: "#/components/schemas/BuildingResponse"
2854         "400":
2855             description: Validation error
2856             content:
2857                 application/json:
2858                     schema:
2859                         $ref: "#/components/schemas/Error"
2860         "401":
2861             description: Unauthorized
2862             content:
2863                 application/json:
2864                     schema:
2865                         $ref: "#/components/schemas/Error"
2866         "403":
2867             description: Forbidden
2868             content:
2869                 application/json:
2870                     schema:
```



```
2871         $ref: "#/components/schemas/Error"
2872     "404":
2873         description: Building not found
2874         content:
2875             application/json:
2876                 schema:
2877                     $ref: "#/components/schemas/Error"
2878     "422":
2879         description: Validation error
2880         content:
2881             application/json:
2882                 schema:
2883                     $ref: "#/components/schemas/Error"
2884     parameters:
2885     - in: path
2886       name: id
2887       schema:
2888           $ref: "#/components/schemas/ObjectId"
2889       required: true
2890     requestBody:
2891       required: true
2892       content:
2893           application/json:
2894               schema:
2895                   $ref: "#/components/schemas/UpdateBuildingRequest"
2896     delete:
2897       operationId: deleteBuildingsById
2898       summary: Delete building
2899       description: Deletes building with associated sensors, readings and alerts
2900       tags:
2901       - Buildings
2902       security:
2903       - bearerAuth: []
2904     responses:
2905     "204":
2906       description: Building and associated data deleted successfully
2907     "400":
2908       description: Invalid ID parameter
2909       content:
2910           application/json:
2911               schema:
2912                   $ref: "#/components/schemas/Error"
2913     "401":
2914       description: Unauthorized
2915       content:
2916           application/json:
2917               schema:
2918                   $ref: "#/components/schemas/Error"
2919     "403":
2920       description: Forbidden
2921       content:
2922           application/json:
2923               schema:
2924                   $ref: "#/components/schemas/Error"
```

```
2925     "404":
2926       description: Building not found
2927       content:
2928         application/json:
2929           schema:
2930             $ref: "#/components/schemas/Error"
2931 parameters:
2932   - in: path
2933     name: id
2934     schema:
2935       $ref: "#/components/schemas/ObjectId"
2936     required: true
2937 /buildings/{id}/readings:
2938   get:
2939     operationId: getBuildingsByIdReadings
2940     summary: Get reading history
2941     description: Returns history for a date range, for charts. Use
2942       limit=1&sortOrder=desc for the latest reading
2943     tags:
2944       - Buildings
2945     security:
2946       - bearerAuth: []
2947     responses:
2948       "200":
2949         description: Historical readings retrieved successfully
2950         content:
2951           application/json:
2952             schema:
2953               $ref: "#/components/schemas/GetBuildingHistoryResponse"
2954       "400":
2955         description: Invalid date range
2956         content:
2957           application/json:
2958             schema:
2959               $ref: "#/components/schemas/Error"
2960       "401":
2961         description: Unauthorized
2962         content:
2963           application/json:
2964             schema:
2965               $ref: "#/components/schemas/Error"
2966       "403":
2967         description: Forbidden
2968         content:
2969           application/json:
2970             schema:
2971               $ref: "#/components/schemas/Error"
2972       "404":
2973         description: Building not found
2974         content:
2975           application/json:
2976             schema:
2977               $ref: "#/components/schemas/Error"
2978 parameters:
```

```

2979     - in: path
2980       name: id
2981       schema:
2982         $ref: "#/components/schemas/ObjectId"
2983       required: true
2984     - in: query
2985       name: startDate
2986       schema:
2987         type: string
2988         format: date-time
2989         pattern: ^(?:?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [048]
           ↪ 00|[13579] [26]00)-02-29|\d{4}-(?:?:0[13578] |1[02])-(?:0[1-9] | [12]
           ↪ \d|3[01])|(?:0[469] |11)-(?:0[1-9] | [12]\d|30)|(?:02)-(?:0[1-9] |1\d|
           ↪ 2[0-8]))T(?:?:[01]\d|2[0-3]):[0-5]\d:[0-5]\d(?:?:\.\d+)?(?:Z))$
2990       required: true
2991       description: Start date for historical data
2992     - in: query
2993       name: endDate
2994       schema:
2995         type: string
2996         format: date-time
2997         pattern: ^(?:?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [048]
           ↪ 00|[13579] [26]00)-02-29|\d{4}-(?:?:0[13578] |1[02])-(?:0[1-9] | [12]
           ↪ \d|3[01])|(?:0[469] |11)-(?:0[1-9] | [12]\d|30)|(?:02)-(?:0[1-9] |1\d|
           ↪ 2[0-8]))T(?:?:[01]\d|2[0-3]):[0-5]\d:[0-5]\d(?:?:\.\d+)?(?:Z))$
2998       required: true
2999       description: End date for historical data
3000     - in: query
3001       name: sensorType
3002       schema:
3003         type: string
3004         enum:
3005           - internal_temp
3006           - external_temp
3007           - energy_meter
3008           - gas_meter
3009       description: Filter by sensor type
3010     - in: query
3011       name: interval
3012       schema:
3013         type: string
3014         enum:
3015           - minute
3016           - hour
3017           - day
3018       description: "Data aggregation interval (default: hour)"
3019     - in: query
3020       name: limit
3021       schema:
3022         type: string
3023       description: "Maximum number of points (1-1000, default: 1000). Use
           ↪ limit=1&sortOrder=desc for the latest point"
3024     - in: query
3025       name: sortOrder

```

```
3027         schema:
3028             default: asc
3029             type: string
3030             enum:
3031                 - asc
3032                 - desc
3033             description: "Sort order by timestamp (default: asc)"
3034 /buildings/{id}/efficiency:
3035     get:
3036         operationId: getBuildingsByIdEfficiency
3037         summary: Calculate efficiency
3038         description: Calculates the building thermal efficiency over the requested
3039             ↪ period
3040         tags:
3041             - Buildings
3042         security:
3043             - bearerAuth: []
3044         responses:
3045             "200":
3046                 description: Efficiency metrics calculated successfully
3047                 content:
3048                     application/json:
3049                         schema:
3050                             $ref: "#/components/schemas/GetBuildingEfficiencyResponse"
3051             "400":
3052                 description: Invalid request parameters
3053                 content:
3054                     application/json:
3055                         schema:
3056                             $ref: "#/components/schemas/Error"
3057             "401":
3058                 description: Unauthorized
3059                 content:
3060                     application/json:
3061                         schema:
3062                             $ref: "#/components/schemas/Error"
3063             "403":
3064                 description: Forbidden
3065                 content:
3066                     application/json:
3067                         schema:
3068                             $ref: "#/components/schemas/Error"
3069             "404":
3070                 description: Building not found
3071                 content:
3072                     application/json:
3073                         schema:
3074                             $ref: "#/components/schemas/Error"
3075         parameters:
3076             - in: path
3077               name: id
3078               schema:
3079                   $ref: "#/components/schemas/ObjectId"
3080             required: true
```

```

3080     - in: query
3081       name: startDate
3082       schema:
3083         type: string
3084         format: date-time
3085         pattern: ^(?:?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [048]
           ↪ 00|[13579] [26]00)-02-29|\d{4}-(?:?:0[13578]|1[02])-(?:0[1-9]| [12]
           ↪ \d|3[01])|(?:0[469]|11)-(?:0[1-9]| [12]\d|30)|(?:02)-(?:0[1-9]|1\d|
           ↪ 2[0-8]))T(?:?:[01]\d|2[0-3]):[0-5]\d:[0-5]\d(?:?:\.\d+)?(?:Z))$
3086       required: true
3087       description: Start date for efficiency calculation
3088     - in: query
3089       name: endDate
3090       schema:
3091         type: string
3092         format: date-time
3093         pattern: ^(?:?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [048]
           ↪ 00|[13579] [26]00)-02-29|\d{4}-(?:?:0[13578]|1[02])-(?:0[1-9]| [12]
           ↪ \d|3[01])|(?:0[469]|11)-(?:0[1-9]| [12]\d|30)|(?:02)-(?:0[1-9]|1\d|
           ↪ 2[0-8]))T(?:?:[01]\d|2[0-3]):[0-5]\d:[0-5]\d(?:?:\.\d+)?(?:Z))$
3094       required: true
3095       description: End date for efficiency calculation
3096 /sensors:
3097   get:
3098     operationId: getSensors
3099     summary: List sensors
3100     description: Returns sensors filtered by building, type and status
3101     tags:
3102       - Sensors
3103     security:
3104       - bearerAuth: []
3105     responses:
3106       "200":
3107         description: Sensors retrieved successfully
3108         content:
3109           application/json:
3110             schema:
3111               $ref: "#/components/schemas/ListSensorsResponse"
3112       "400":
3113         description: Invalid query parameters
3114         content:
3115           application/json:
3116             schema:
3117               $ref: "#/components/schemas/Error"
3118   parameters:
3119     - in: query
3120       name: limit
3121       schema:
3122         type: string
3123         description: "Maximum number of results (1-100, default: 50)"
3124     - in: query
3125       name: offset
3126       schema:
3127         type: string

```

```
3128     description: "Number of results to skip (default: 0)"
3129   - in: query
3130     name: building
3131     schema:
3132       $ref: "#/components/schemas/ObjectId"
3133   - in: query
3134     name: sensorType
3135     schema:
3136       type: string
3137       enum:
3138         - internal_temp
3139         - external_temp
3140         - energy_meter
3141         - gas_meter
3142     description: Filter by sensor type
3143   - in: query
3144     name: status
3145     schema:
3146       type: string
3147       enum:
3148         - active
3149         - inactive
3150         - maintenance
3151         - error
3152     description: Filter by sensor status
3153   - in: query
3154     name: sortBy
3155     schema:
3156       type: string
3157       enum:
3158         - createdAt
3159         - updatedAt
3160         - sensorType
3161         - status
3162     description: "Field to sort by (default: createdAt)"
3163   - in: query
3164     name: sortOrder
3165     schema:
3166       type: string
3167       enum:
3168         - asc
3169         - desc
3170     description: "Sort order (default: desc)"
3171 post:
3172   operationId: postSensors
3173   summary: Create sensor
3174   description: Creates a new sensor for an existing building
3175   tags:
3176     - Sensors
3177   security:
3178     - bearerAuth: []
3179   responses:
3180     "201":
3181       description: Sensor created successfully
```

```
3182         content:
3183             application/json:
3184                 schema:
3185                     $ref: "#/components/schemas/SensorResponse"
3186 "400":
3187     description: Validation error
3188     content:
3189         application/json:
3190             schema:
3191                 $ref: "#/components/schemas/Error"
3192 "404":
3193     description: Building not found
3194     content:
3195         application/json:
3196             schema:
3197                 $ref: "#/components/schemas/Error"
3198 "409":
3199     description: Sensor with this serial number already exists
3200     content:
3201         application/json:
3202             schema:
3203                 $ref: "#/components/schemas/Error"
3204 requestBody:
3205     required: true
3206     content:
3207         application/json:
3208             schema:
3209                 $ref: "#/components/schemas/CreateSensorInput"
3210 /sensors/{id}:
3211     get:
3212         operationId: getSensorsById
3213         summary: Get sensor
3214         description: Returns sensor detail with building data
3215         tags:
3216             - Sensors
3217         security:
3218             - bearerAuth: []
3219         responses:
3220             "200":
3221                 description: Sensor details retrieved successfully
3222                 content:
3223                     application/json:
3224                         schema:
3225                             $ref: "#/components/schemas/SensorResponse"
3226             "400":
3227                 description: Invalid ID parameter
3228                 content:
3229                     application/json:
3230                         schema:
3231                             $ref: "#/components/schemas/Error"
3232             "404":
3233                 description: Sensor not found
3234                 content:
3235                     application/json:
```

```
3236         schema:
3237             $ref: "#/components/schemas/Error"
3238     parameters:
3239         - in: path
3240           name: id
3241           schema:
3242             $ref: "#/components/schemas/ObjectId"
3243           required: true
3244     patch:
3245         operationId: patchSensorsById
3246         summary: Update sensor
3247         description: Updates sensor configuration and thresholds
3248         tags:
3249             - Sensors
3250         security:
3251             - bearerAuth: []
3252         responses:
3253             "200":
3254                 description: Sensor updated successfully
3255                 content:
3256                     application/json:
3257                         schema:
3258                             $ref: "#/components/schemas/SensorResponse"
3259             "400":
3260                 description: Validation error
3261                 content:
3262                     application/json:
3263                         schema:
3264                             $ref: "#/components/schemas/Error"
3265             "404":
3266                 description: Sensor not found
3267                 content:
3268                     application/json:
3269                         schema:
3270                             $ref: "#/components/schemas/Error"
3271             "409":
3272                 description: Sensor with this serial number already exists
3273                 content:
3274                     application/json:
3275                         schema:
3276                             $ref: "#/components/schemas/Error"
3277         parameters:
3278             - in: path
3279               name: id
3280               schema:
3281                 $ref: "#/components/schemas/ObjectId"
3282               required: true
3283         requestBody:
3284             required: true
3285             content:
3286                 application/json:
3287                     schema:
3288                         $ref: "#/components/schemas/UpdateSensorRequest"
3289     delete:
```



```
3290     operationId: deleteSensorsById
3291     summary: Delete sensor
3292     description: Deletes sensor with associated readings and alerts
3293     tags:
3294       - Sensors
3295     security:
3296       - bearerAuth: []
3297     responses:
3298       "204":
3299         description: Sensor deleted successfully
3300       "400":
3301         description: Invalid ID parameter
3302         content:
3303           application/json:
3304             schema:
3305               $ref: "#/components/schemas/Error"
3306       "404":
3307         description: Sensor not found
3308         content:
3309           application/json:
3310             schema:
3311               $ref: "#/components/schemas/Error"
3312     parameters:
3313       - in: path
3314         name: id
3315         schema:
3316           $ref: "#/components/schemas/ObjectId"
3317         required: true
3318 /sensors/{id}/readings:
3319   get:
3320     operationId: getSensorsByIdReadings
3321     summary: Get sensor readings
3322     description: Returns sensor reading history with pagination
3323     tags:
3324       - Sensors
3325     security:
3326       - bearerAuth: []
3327     responses:
3328       "200":
3329         description: Readings retrieved successfully
3330         content:
3331           application/json:
3332             schema:
3333               $ref: "#/components/schemas/GetSensorReadingsResponse"
3334       "400":
3335         description: Invalid query parameters
3336         content:
3337           application/json:
3338             schema:
3339               $ref: "#/components/schemas/Error"
3340       "404":
3341         description: Sensor not found
3342         content:
3343           application/json:
```

```

3344         schema:
3345             $ref: "#/components/schemas/Error"
3346     parameters:
3347         - in: path
3348           name: id
3349           schema:
3350             $ref: "#/components/schemas/ObjectId"
3351           required: true
3352         - in: query
3353           name: limit
3354           schema:
3355             type: string
3356             description: "Maximum number of results (1-100, default: 50)"
3357         - in: query
3358           name: offset
3359           schema:
3360             type: string
3361             description: "Number of results to skip (default: 0)"
3362         - in: query
3363           name: startDate
3364           schema:
3365             type: string
3366             format: date-time
3367             pattern: ^((?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [048]
3368               ↪ 00|[13579] [26]00)-02-29|\d{4}-(?: (?:0[13578] |1[02])-(?:0[1-9] | [12]
3369               ↪ \d|3[01])|(?:0[469] |11)-(?:0[1-9] | [12]\d|30)|(?:02)-(?:0[1-9] |1\d|
3370               ↪ 2[0-8]))T(?: (?:[01]\d|2[0-3]):[0-5]\d:[0-5]\d(?:\.\d+)?(?:Z))$
3371             description: Start date for filtering readings (ISO 8601)
3372         - in: query
3373           name: endDate
3374           schema:
3375             type: string
3376             format: date-time
3377             pattern: ^((?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [048]
3378               ↪ 00|[13579] [26]00)-02-29|\d{4}-(?: (?:0[13578] |1[02])-(?:0[1-9] | [12]
3379               ↪ \d|3[01])|(?:0[469] |11)-(?:0[1-9] | [12]\d|30)|(?:02)-(?:0[1-9] |1\d|
3380               ↪ 2[0-8]))T(?: (?:[01]\d|2[0-3]):[0-5]\d:[0-5]\d(?:\.\d+)?(?:Z))$
3381             description: End date for filtering readings (ISO 8601)
3382         - in: query
3383           name: sortOrder
3384           schema:
3385             type: string
3386             enum:
3387               - asc
3388               - desc
3389             description: "Sort order by timestamp (default: desc)"
3390 /alerts:
3391     get:
3392         operationId: getAlerts
3393         tags:
3394             - Alerts
3395         summary: List alerts
3396         description: Returns paginated alerts with optional filters
3397         responses:

```

```
3392     "200":
3393         description: List of alerts retrieved successfully
3394         content:
3395             application/json:
3396                 schema:
3397                     $ref: "#/components/schemas/ListAlertsResponse"
3398     "400":
3399         description: Validation error
3400         content:
3401             application/json:
3402                 schema:
3403                     $ref: "#/components/schemas/Error"
3404     "422":
3405         description: Invalid query parameters
3406         content:
3407             application/json:
3408                 schema:
3409                     $ref: "#/components/schemas/Error"
3410 parameters:
3411     - in: query
3412       name: limit
3413       schema:
3414         type: string
3415         description: "Maximum number of results (1-100, default: 50)"
3416     - in: query
3417       name: offset
3418       schema:
3419         type: string
3420         description: "Number of results to skip (default: 0)"
3421     - in: query
3422       name: building
3423       schema:
3424         $ref: "#/components/schemas/ObjectId"
3425     - in: query
3426       name: status
3427       schema:
3428         type: string
3429         enum:
3430             - active
3431             - acknowledged
3432             - resolved
3433         description: Filter by alert status
3434     - in: query
3435       name: severity
3436       schema:
3437         type: string
3438         enum:
3439             - low
3440             - medium
3441             - high
3442             - critical
3443         description: Filter by severity
3444     - in: query
3445       name: sortBy
```

```
3446     schema:
3447       type: string
3448     enum:
3449       - createdAt
3450       - updatedAt
3451       - severity
3452       - status
3453     description: "Field to sort by (default: createdAt)"
3454 - in: query
3455   name: sortOrder
3456   schema:
3457     type: string
3458     enum:
3459       - asc
3460       - desc
3461     description: "Sort order (default: desc)"
3462 /alerts/{id}:
3463   get:
3464     operationId: getAlertsById
3465     tags:
3466       - Alerts
3467     summary: Get alert
3468     description: Returns single alert detail by ID
3469     security:
3470       - bearerAuth: []
3471     responses:
3472       "200":
3473         description: Alert retrieved successfully
3474         content:
3475           application/json:
3476             schema:
3477               $ref: "#/components/schemas/AlertResponse"
3478       "400":
3479         description: Validation error
3480         content:
3481           application/json:
3482             schema:
3483               $ref: "#/components/schemas/Error"
3484       "401":
3485         description: Unauthorized
3486         content:
3487           application/json:
3488             schema:
3489               $ref: "#/components/schemas/Error"
3490       "403":
3491         description: Forbidden
3492         content:
3493           application/json:
3494             schema:
3495               $ref: "#/components/schemas/Error"
3496       "404":
3497         description: Alert not found
3498         content:
3499           application/json:
```

```
3500         schema:
3501             $ref: "#/components/schemas/Error"
3502     "422":
3503         description: Invalid alert ID format
3504         content:
3505             application/json:
3506                 schema:
3507                     $ref: "#/components/schemas/Error"
3508     parameters:
3509     - in: path
3510       name: id
3511       schema:
3512           $ref: "#/components/schemas/ObjectId"
3513       required: true
3514     patch:
3515         operationId: patchAlertsById
3516         tags:
3517         - Alerts
3518         summary: Update alert status
3519         description: Updates an alert status (acknowledge or resolve)
3520         responses:
3521             "200":
3522                 description: Alert status updated successfully
3523                 content:
3524                     application/json:
3525                         schema:
3526                             $ref: "#/components/schemas/AlertResponse"
3527             "400":
3528                 description: Validation error
3529                 content:
3530                     application/json:
3531                         schema:
3532                             $ref: "#/components/schemas/Error"
3533             "404":
3534                 description: Alert not found
3535                 content:
3536                     application/json:
3537                         schema:
3538                             $ref: "#/components/schemas/Error"
3539             "409":
3540                 description: Invalid alert status transition (already resolved or not
3541                 ↳ active)
3542                 content:
3543                     application/json:
3544                         schema:
3545                             $ref: "#/components/schemas/Error"
3546             "422":
3547                 description: Invalid alert ID format
3548                 content:
3549                     application/json:
3550                         schema:
3551                             $ref: "#/components/schemas/Error"
3552     parameters:
3553     - in: path
```

```
3553     name: id
3554     schema:
3555       $ref: "#/components/schemas/ObjectId"
3556     required: true
3557   requestBody:
3558     required: true
3559     content:
3560       application/json:
3561         schema:
3562           $ref: "#/components/schemas/UpdateAlertStatusRequest"
3563 /metrics:
3564   get:
3565     operationId: getMetrics
3566     summary: Get system metrics
3567     description: "Returns aggregated KPIs: sensors, active alerts and
3568       ↪ consumption"
3569     tags:
3570       - Metrics
3571     security:
3572       - bearerAuth: []
3573     responses:
3574       "200":
3575         description: System metrics retrieved successfully
3576         content:
3577           application/json:
3578             schema:
3579               $ref: "#/components/schemas/MetricsResponse"
3580       "401":
3581         description: Unauthorized
3582         content:
3583           application/json:
3584             schema:
3585               $ref: "#/components/schemas/Error"
3586 /metrics/timeseries:
3587   get:
3588     operationId: getMetricsTimeseries
3589     summary: Get metrics timeseries
3590     description: Returns aggregated electricity and gas per interval
3591       ↪ (hour/day/week)
3592     tags:
3593       - Metrics
3594     security:
3595       - bearerAuth: []
3596     responses:
3597       "200":
3598         description: Historical metrics data retrieved successfully
3599         content:
3600           application/json:
3601             schema:
3602               $ref: "#/components/schemas/MetricsHistoryResponse"
3603       "400":
3604         description: Validation error
3605         content:
3606           application/json:
```

```

3605         schema:
3606             $ref: "#/components/schemas/Error"
3607     "401":
3608         description: Unauthorized
3609         content:
3610             application/json:
3611                 schema:
3612                     $ref: "#/components/schemas/Error"
3613     "422":
3614         description: Invalid query parameters
3615         content:
3616             application/json:
3617                 schema:
3618                     $ref: "#/components/schemas/Error"
3619 parameters:
3620     - in: query
3621       name: startDate
3622       schema:
3623         anyOf:
3624         - type: string
3625           format: date-time
3626           pattern: ^(?:?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [
3627             ↪ 048]00|[13579] [26]00)-02-29|\d{4}-(?:?:0[13578] |1[02])-(?:0[1
3628             ↪ -9] | [12]\d|3[01])|(?:0[469] |11)-(?:0[1-9] | [12]\d|30)|(?:02)-(?:
3629             ↪ :0[1-9] |1\d|2[0-8]))T(?:?: [01]\d|2[0-3]):[0-5]\d:[0-5]\d(?::\
3630             ↪ .\d+)?(?:Z|([+-](?: [01]\d|2[0-3]):[0-5]\d)))$
3631         - type: string
3632           format: date
3633           pattern: ^(?:?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [
3634             ↪ 048]00|[13579] [26]00)-02-29|\d{4}-(?:?:0[13578] |1[02])-(?:0[1
3635             ↪ -9] | [12]\d|3[01])|(?:0[469] |11)-(?:0[1-9] | [12]\d|30)|(?:02)-(?:
3636             ↪ :0[1-9] |1\d|2[0-8]))$
3637       required: true
3638       description: Start of the time range (ISO 8601)
3639     - in: query
3640       name: endDate
3641       schema:
3642         anyOf:
3643         - type: string
3644           format: date-time
3645           pattern: ^(?:?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [
3646             ↪ 048]00|[13579] [26]00)-02-29|\d{4}-(?:?:0[13578] |1[02])-(?:0[1
3647             ↪ -9] | [12]\d|3[01])|(?:0[469] |11)-(?:0[1-9] | [12]\d|30)|(?:02)-(?:
3648             ↪ :0[1-9] |1\d|2[0-8]))T(?:?: [01]\d|2[0-3]):[0-5]\d:[0-5]\d(?::\
3649             ↪ .\d+)?(?:Z|([+-](?: [01]\d|2[0-3]):[0-5]\d)))$
3650         - type: string
3651           format: date
3652           pattern: ^(?:?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [
3653             ↪ 048]00|[13579] [26]00)-02-29|\d{4}-(?:?:0[13578] |1[02])-(?:0[1
3654             ↪ -9] | [12]\d|3[01])|(?:0[469] |11)-(?:0[1-9] | [12]\d|30)|(?:02)-(?:
3655             ↪ :0[1-9] |1\d|2[0-8]))$
3656       required: true
3657       description: End of the time range (ISO 8601)
3658     - in: query

```

```
3645     name: interval
3646     schema:
3647       default: day
3648       type: string
3649       enum:
3650         - hour
3651         - day
3652         - week
3653       description: "Bucket interval for aggregation (default: day)"
3654 /settings:
3655   get:
3656     operationId: getSettings
3657     summary: Get configuration
3658     description: Returns the current system configuration
3659     tags:
3660       - Settings
3661     security:
3662       - bearerAuth: []
3663     responses:
3664       "200":
3665         description: System configuration retrieved successfully
3666         content:
3667           application/json:
3668             schema:
3669               $ref: "#/components/schemas/SettingsResponse"
3670       "401":
3671         description: Unauthorized
3672         content:
3673           application/json:
3674             schema:
3675               $ref: "#/components/schemas/Error"
3676       "403":
3677         description: Forbidden - requires admin or operator role
3678         content:
3679           application/json:
3680             schema:
3681               $ref: "#/components/schemas/Error"
3682   patch:
3683     operationId: patchSettings
3684     summary: Update configuration
3685     description: Updates polling, notifications and retention (admin only)
3686     tags:
3687       - Settings
3688     security:
3689       - bearerAuth: []
3690     responses:
3691       "200":
3692         description: System configuration updated successfully
3693         content:
3694           application/json:
3695             schema:
3696               $ref: "#/components/schemas/SettingsResponse"
3697       "400":
3698         description: Validation error
```



```
3699         content:
3700             application/json:
3701                 schema:
3702                     $ref: "#/components/schemas/Error"
3703     "401":
3704         description: Unauthorized
3705         content:
3706             application/json:
3707                 schema:
3708                     $ref: "#/components/schemas/Error"
3709     "403":
3710         description: Forbidden - requires admin role
3711         content:
3712             application/json:
3713                 schema:
3714                     $ref: "#/components/schemas/Error"
3715     requestBody:
3716         required: true
3717         content:
3718             application/json:
3719                 schema:
3720                     $ref: "#/components/schemas/UpdateSettingsRequest"
3721 /readings:
3722     get:
3723         operationId: getReadings
3724         summary: Export readings
3725         description: Exports filtered readings for buildings and period as CSV or
3726         ↪ JSON
3727         (admin only)
3728     tags:
3729         - Readings
3730     security:
3731         - bearerAuth: []
3732     responses:
3733         "200":
3734             description: Readings in requested format (CSV or JSON)
3735             content:
3736                 text/csv:
3737                     schema:
3738                         type: string
3739                 application/json:
3740                     schema:
3741                         type: object
3742         "400":
3743             description: Invalid query parameters
3744             content:
3745                 application/json:
3746                     schema:
3747                         $ref: "#/components/schemas/Error"
3748         "401":
3749             description: Unauthorized
3750             content:
3751                 application/json:
3752                     schema:
```

```

3752         $ref: "#/components/schemas/Error"
3753     "403":
3754         description: Forbidden - requires admin role
3755         content:
3756             application/json:
3757                 schema:
3758                     $ref: "#/components/schemas/Error"
3759     "404":
3760         description: One or more buildings were not found
3761         content:
3762             application/json:
3763                 schema:
3764                     $ref: "#/components/schemas/Error"
3765 parameters:
3766     - in: query
3767       name: buildingIds
3768       schema:
3769         type: string
3770         minLength: 1
3771         description: Comma-separated list of building IDs
3772     - in: query
3773       name: startDate
3774       schema:
3775         type: string
3776         format: date
3777         pattern: ^(?:?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [048]
3778           ↪ 00|[13579] [26]00)-02-29|\d{4}-(?:?:0[13578] |1[02])-(?:0[1-9] | [12]
3779           ↪ \d|3[01])|(?:0[469] |11)-(?:0[1-9] | [12]\d|30)|(?:02)-(?:0[1-9] |1\d|
3780           ↪ 2[0-8]))))$
3781         description: First date to include (YYYY-MM-DD)
3782     - in: query
3783       name: endDate
3784       schema:
3785         type: string
3786         format: date
3787         pattern: ^(?:?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [048]
3788           ↪ 00|[13579] [26]00)-02-29|\d{4}-(?:?:0[13578] |1[02])-(?:0[1-9] | [12]
3789           ↪ \d|3[01])|(?:0[469] |11)-(?:0[1-9] | [12]\d|30)|(?:02)-(?:0[1-9] |1\d|
3790           ↪ 2[0-8]))))$
3791         description: Last date to include (YYYY-MM-DD)
3792     - in: query
3793       name: format
3794       schema:
3795         type: string
3796         enum:
3797           - csv
3798           - json
3799       description: Representation format
3800 /reports:
3801   get:
3802     operationId: getReports
3803     summary: Download energy report
3804     description: Generates the aggregated report for buildings and period as PDF
3805     ↪ or

```

```
3799     Excel (admin only)
3800 tags:
3801   - Reports
3802 security:
3803   - bearerAuth: []
3804 responses:
3805   "200":
3806     description: Report file (PDF or Excel) containing aggregated consumption
3807     ↪ data
3808     content:
3809       application/pdf:
3810         schema:
3811           type: string
3812           format: binary
3813       application/vnd.openxmlformats-officedocument.spreadsheetml.sheet:
3814         schema:
3815           type: string
3816           format: binary
3817   "400":
3818     description: Invalid report parameters
3819     content:
3820       application/json:
3821         schema:
3822           $ref: "#/components/schemas/Error"
3823   "401":
3824     description: Unauthorized
3825     content:
3826       application/json:
3827         schema:
3828           $ref: "#/components/schemas/Error"
3829   "403":
3830     description: Forbidden - requires admin role
3831     content:
3832       application/json:
3833         schema:
3834           $ref: "#/components/schemas/Error"
3835   "404":
3836     description: One or more buildings were not found
3837     content:
3838       application/json:
3839         schema:
3840           $ref: "#/components/schemas/Error"
3841 parameters:
3842   - in: query
3843     name: buildingIds
3844     schema:
3845       type: string
3846       minLength: 1
3847     required: true
3848     description: Comma-separated list of building IDs to include in the
3849     ↪ report
3850   - in: query
3851     name: startDate
3852     schema:
```

```

3851         type: string
3852         format: date
3853         pattern: ^(?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [048]
    ↪ 00|[13579] [26]00)-02-29|\d{4}-(?:0[13578]|1[02])-(?:0[1-9]| [12]
    ↪ \d|3[01])|(?:0[469]|11)-(?:0[1-9]| [12]\d|30)|(?:02)-(?:0[1-9]|1\d|
    ↪ 2[0-8]))))$
3854     required: true
3855     description: First date to include (YYYY-MM-DD)
3856 - in: query
3857     name: endDate
3858     schema:
3859         type: string
3860         format: date
3861         pattern: ^(?:\d\d[2468] [048]|\d\d[13579] [26]|\d\d0[48] | [02468] [048]
    ↪ 00|[13579] [26]00)-02-29|\d{4}-(?:0[13578]|1[02])-(?:0[1-9]| [12]
    ↪ \d|3[01])|(?:0[469]|11)-(?:0[1-9]| [12]\d|30)|(?:02)-(?:0[1-9]|1\d|
    ↪ 2[0-8]))))$
3862     required: true
3863     description: Last date to include (YYYY-MM-DD)
3864 - in: query
3865     name: format
3866     schema:
3867         default: pdf
3868         type: string
3869         enum:
3870             - pdf
3871             - xlsx
3872     description: Report file format (pdf or xlsx)
3873 /backups:
3874     get:
3875         operationId: getBackups
3876         summary: Download backup
3877         description: Generates and downloads the full JSON database backup (admin
    ↪ only)
3878         tags:
3879             - Backups
3880         security:
3881             - bearerAuth: []
3882         responses:
3883             "200":
3884                 description: JSON backup file download
3885                 content:
3886                     application/json:
3887                         schema:
3888                             type: object
3889             "401":
3890                 description: Unauthorized
3891                 content:
3892                     application/json:
3893                         schema:
3894                             $ref: "#/components/schemas/Error"
3895             "403":
3896                 description: Forbidden - requires admin role
3897                 content:

```

3898

3899

3900

```
application/json:
  schema:
    $ref: "#/components/schemas/Error"
```

2 Implementazione

L'applicazione è stata sviluppata adottando un'architettura client-server disaccoppiata (API RESTful e Single Page Application per il frontend) e utilizzando il linguaggio di programmazione TypeScript sia per il frontend che per il backend. Per la parte di backend è stato sfruttato il runtime BunJS con il framework Hono per la creazione e la gestione del server e delle API RESTful. Per la parte di frontend è stato utilizzato il framework React (versione 19) con il bundler Vite. Il database utilizzato è MongoDB, gestito tramite l'ODM Mongoose.

L'acquisizione dei dati dai sensori IoT avviene tramite il protocollo MQTT: il backend si sottoscrive al broker (topic `sensors/sensorId/readings`) e archivia le rilevazioni ricevute in MongoDB, utilizzando una time-series collection per lo storico delle misurazioni.

2.1 Scelta del Tech Stack e Motivazioni Architettureali

La scelta di usare TypeScript è scaturita dalla necessità di avere un controllo maggiore sul type-checking e una maggiore manutenibilità del codice. Questo ci permette di catturare una serie di bug a compile-time, evitando errori di tipo durante l'esecuzione.

La scelta del framework Hono presenta alcune principali motivazioni: performance, portabilità e Developer Experience (DX). Hono garantisce performance migliori rispetto a Express.js e risulta conforme agli standard Web API, facendo sì che si possa fare l'hosting anche su piattaforme edge (come Cloudflare Workers o Vercel functions). Inoltre Hono presenta un client per il frontend tipizzato, così che le risposte delle API siano tipizzate end-to-end. Così facendo si ottiene l'auto-completamento e convalida dei parametri e dei payload restituiti a compile-time, eliminando il rischio di disallineamenti tra i due layer.

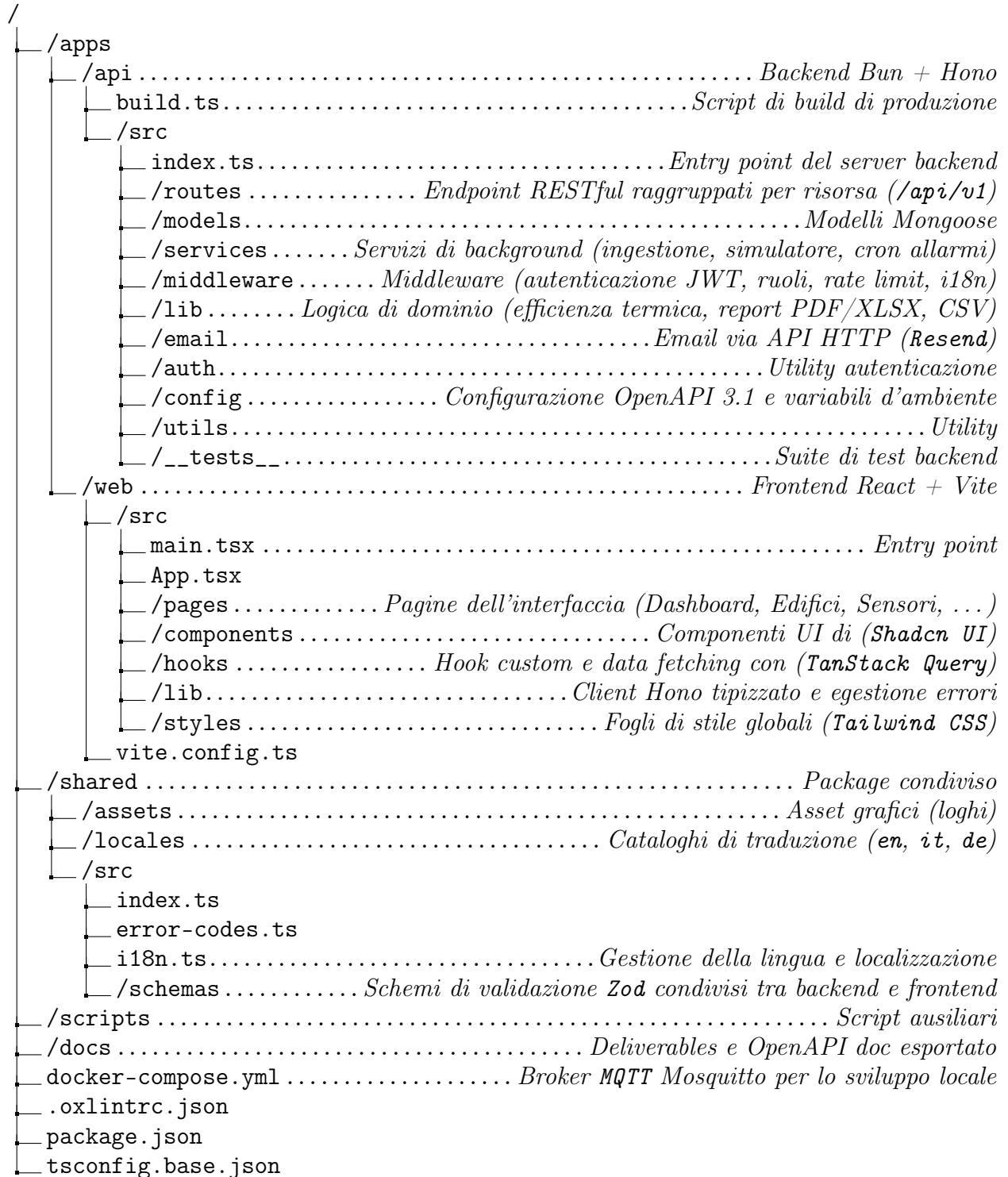
Il runtime BunJS è stato scelto principalmente per l'ottima DX e per le sue performance. Bun esegue nativamente TypeScript senza aver bisogno di strumenti come ts-node o nodemon. Inoltre esso integra un package manager molto più prestante di npm. Infine, l'inclusione di alcune API (come cron e hashing per le password) ha ridotto la necessità di scaricare altre dipendenze esterne.

Per il frontend, è stato adottato React 19 data l'esigenza di realizzare una Single Page Application. Rispetto ad alternative come Vue.js, la preferenza per React è stata guidata dalla maggiore capillarità dell'ecosistema e dall'integrazione con librerie come TanStack Query, React Router e Shadcn UI (quest'ultima basata sulle primitive React). Questo ha reso la gestione di caching, UI e routing molto più semplice. Inoltre l'affiancamento a Vite come build tool consente di ottenere un HMR istantaneo, migliorando ulteriormente la DX.

La persistenza dei dati si basa su MongoDB e sull'ODM Mongoose. L'uso di MongoDB è stato giustificato in particolare dalla presenza nativa delle Time-Series Collections, permettendo di gestire grandi volumi di dati di time-series, come le letture dei sensori. Ciò è risultato essenziale per il calcolo dell'efficienza di ogni edificio in maniera efficiente. Infine, l'acquisizione dei dati mediante protocollo MQTT risponde ai vincoli operativi delle architetture IoT. A differenza di HTTP, MQTT opera tramite un modello Publish/Subscribe su TCP rendendolo essenziale per la gestione dei dati provenienti dai sensori.

2.2 Repository Organization

Il codice del progetto (github.com/alterax05/wattguard-ing-sw) è organizzato secondo la seguente struttura:



2.3 Branching Strategy e organizzazione del lavoro

Per la gestione e condivisione del codice si è scelto di utilizzare GitHub e di adottare una strategia di branching basata su *GitFlow*. In particolare si è deciso di utilizzare i seguenti *branch*:

main *Branch* principale, contiene il codice stabile;

`develop` *Branch* di integrazione, sul quale confluiscono tutte le funzionalità sviluppate;

`feature/<nome>` *Branch* per lo sviluppo di nuove funzionalità, creati da `develop`

`fix/<nome>` *Branch* per le correzioni di errori

`docs/<nome>` *Branch* per la documentazione.

I commit seguono la specifica Conventional Commits (`feat:`, `fix:`, `chore:`, ...). Ad ogni push sul ramo `develop` e sul ramo `main` vengono eseguite verifiche sulla qualità del codice tra cui controlli statici e test.

Per la divisione delle aree di lavoro abbiamo individuato un responsabile principale mentre per le slide del pitch ognuno ha contribuito in modo equo.

- **Michele Porcellato:** responsabile parte pratica del backend. Responsabile principale della scrittura del documento D2 e addetto alla prima scrittura dei requisiti funzionali e non funzionali per il deliverable D1.
- **Marco Piovesan:** Responsabile principale per il documento D1 e frontend. Inoltre è corresponsabile per documento D3.
- **Alessio Pesaresi:** Responsabile principale per il documento D3, addetto alla stesura del business plan e alla realizzazione del video. Inoltre è corresponsabile per lo sviluppo del frontend.

2.4 *Dependency*

Dipendenze backend e condivise Per il *backend* e il pacchetto condiviso sono state impiegati i seguenti moduli esterni:

`hono framework` web per la gestione delle API;

`hono-openapi` per la generazione automatica della specifica `OpenAPI 3.1` partendo dalle definizioni delle rotte;

`@scalar/hono-api-reference` per la documentazione interattiva (Scalar UI) esposta all'indirizzo `/api/v1/docs`;

`hono-rate-limiter` per la protezione da attacchi di brute-force (autenticazione, recupero password, inviti);

`mongoose` libreria ODM (Object Data Modeling) per l'interazione con il database `MongoDB`;

`mqtt` client MQTT per la connessione al broker e l'acquisizione delle letture dei sensori IoT;

`zod` libreria per la validazione dei dati in input;

`resend` per l'invio di email tramite API HTTP;

`google-auth-library` per supporto del login con Google;

`exceljs` e `pdfkit` per la generazione dei report di efficienza energetica in formato `.xlsx` e `.pdf`;

`i18next` per il supporto multilingua sia sul frontend sia sul backend;

Dipendenze frontend Per il *frontend* le librerie principali includono:

`react`, `react-dom` libreria frontend React;

`vite` bundler e dev server;

`react-router-dom` per la gestione del routing client-side;

`@tanstack/react-query` per la gestione di fetching, caching, e sincronizzazione di dati con il backend;

`tailwindcss (v4)` e `radix-ui` per la gestione dello stile dell'applicazione;

`recharts` per la visualizzazione di grafici;

`leaflet` e `react-leaflet` per la mappa degli edifici monitorati;

`sonner` e `next-themes` per il sistema di notifiche *toast* e la gestione del tema chiaro e scuro;

`react-i18next` e `i18next-browser-languagedetector` per l'internazionalizzazione dell'interfaccia;

`react-hook-form` e `@hookform/resolvers` per la gestione dei moduli presenti nel frontend e la validazione integrata tramite Zod;

`date-fns` per la gestione di date e intervalli temporali;

`lucide-react` catalogo completo di icone vettoriali per l'interfaccia utente;

Dipendenze di sviluppo

`typescript` per il controllo statico dei tipi;

`oxlint` linter per l'analisi statica del codice;

`mongodb-memory-server` per l'avvio di un'istanza MongoDB effimera durante l'esecuzione dei test di integrazione;

2.5 Database

Il database è stato progettato per contenere le seguenti collezioni su MongoDB, gestite tramite Mongoose:

2.5.1 Utente

La collezione `users` viene usata per memorizzare i dati relativi agli utenti del sistema. Ogni documento della collezione contiene i seguenti campi:

`_id` (`ObjectId`) identificativo univoco dell'utente;

`email` (`String`) email dell'utente, univoca;

`name` (`String`) nome visualizzato dell'utente (facoltativo);

`role` (`String`, `enum admin|operator`) ruolo dell'utente nel sistema;

`isDisabled` (`Boolean`) *flag* che indica se l'account è disabilitato;

`language` (String, enum en|it|de) preferenza linguistica dell'utente (facoltativa);
`passwordHash` (String) hash bcrypt della password;
`googleSub` (String) identificativo univoco Google
`lastLoginAt` (Date) timestamp dell'ultimo accesso effettuato;
`passwordUpdatedAt` (Date) data dell'ultimo cambio password;
`createdAt` / `updatedAt` (Date) timestamp di creazione e modifica;

2.5.2 Invito

La collezione `invites` viene usata per memorizzare gli inviti inviati via email per la registrazione dei nuovi utenti. Ogni documento contiene i seguenti campi:

`_id` (ObjectId) identificativo univoco dell'invito;
`email` (String) email dell'invitato
`role` (String, enum admin|operator) ruolo assegnato all'utente al momento dell'accettazione;
`tokenHash` (String) hash SHA-256 del token segreto generato casualmente;
`status` (String, enum pending|accepted|revoked|expired) stato corrente dell'invito;
`expiresAt` (Date) scadenza dell'invito (7 giorni dalla creazione), associata a un indice **TTL** per la cancellazione automatica;
`createdBy` (ObjectId) amministratore che ha generato l'invito (riferimento a `User`);
`acceptedAt` (Date) data e ora di avvenuta accettazione dell'invito;
`createdAt` / `updatedAt` (Date) timestamp di creazione e modifica;

2.5.3 TokenResetPassword

La collezione `passwordresettokens` viene usata per memorizzare i token monouso utilizzati durante il reset della password. Ogni documento contiene i seguenti campi:

`_id` (ObjectId) identificativo univoco del token;
`userId` (ObjectId) utente a cui il token è associato (riferimento a `User`);
`tokenHash` (String) hash SHA-256 del token monouso, univoco;
`expiresAt` (Date) scadenza del token (1 ora dalla richiesta), associata a un indice **TTL** per la rimozione automatica;
`usedAt` (Date) data e ora di effettivo utilizzo, al fine di prevenire attacchi di riuso del token;
`createdAt` / `updatedAt` (Date) timestamp di creazione e modifica;

2.5.4 Edificio

La collezione `buildings` viene usata per memorizzare l'anagrafica, i parametri termici e la localizzazione degli edifici monitorati. Ogni documento contiene i seguenti campi:

`_id` (`ObjectId`) identificativo univoco dell'edificio;

`name` (`String`) nome identificativo dell'edificio;

`address` (`String`) indirizzo stradale completo;

`surface` (`Number`) superficie utile riscaldata in m^2 ;

`ceilingHeight` (`Number`) altezza media dei soffitti in metri (minimo 0.5, massimo 20, default 3.0);

`location` (`Object`) posizione geografica GeoJSON `{type: 'Point', coordinates: [lng, lat]}`, indicizzata con indice geospaziale sferico `2dsphere`;

`buildingType` (`ObjectId`) riferimento alla tipologia di edificio (riferimento a `BuildingType`);

`heatingSystemType` (`String`) tipo di impianto termico installato (es. pompa di calore, teleriscaldamento, caldaia a gas);

`constructionYear` (`Number`) anno di costruzione dell'immobile;

`geographicZone` (`String`) zona geografica o climatica dell'edificio;

`status` (`String`, enum `active|inactive|decommissioned`) stato operativo dell'edificio;

`efficiencyThresholds` (`Object`) configurazione dei limiti di efficienza energetica `{enabled: Boolean, minCOP: Number}`;

`createdBy` / `updatedBy` (`ObjectId`) riferimenti agli utenti che hanno creato e aggiornato l'edificio;

`createdAt` / `updatedAt` (`Date`) timestamp di creazione e modifica;

.

2.5.5 TipologiaEdificio

La collezione `buildingtypes` viene usata per memorizzare le categorie di edifici per la ricerca. Ogni documento contiene i seguenti campi:

`_id` (`ObjectId`) identificativo univoco della tipologia;

`name` (`String`) nome della tipologia, univoco nel sistema;

`description` (`String`) descrizione delle caratteristiche della tipologia (facoltativa);

`createdAt` / `updatedAt` (`Date`) timestamp di creazione e modifica;

2.5.6 Sensore

La collezione `sensors` viene usata per memorizzare i sensori installati negli edifici, le eventuali soglie di allerta e lo stato di operatività. Ogni documento contiene i seguenti campi:

`_id` (`ObjectId`) identificativo univoco del sensore;

`building` (`ObjectId`) riferimento all'edificio in cui il sensore è installato (indicizzato, ref `Building`);

`sensorType` (`String`, enum `internal_temp|external_temp|energy_meter|gas_meter`) tipo di sensore supportato dall'applicazione;

`location` (`String`) collocazione fisica del sensore nell'edificio;

`serialNumber` (`String`) numero di serie hardware del sensore;

`installationDate` (`Date`) data di installazione del dispositivo (default `Date.now`);

`status` (`String`, enum `active|inactive|maintenance|error`) stato operativo del sensore;

`lastReading` (`Object`) ultima rilevazione ricevuta `{value, timestamp, unit}`, denormalizzata per la lettura istantanea senza interrogare la collezione `sensorreadings`;

`transmissionInterval` (`Number`) intervallo nominale di trasmissione in secondi;

`minThreshold` / `maxThreshold` (`Number`) soglie minima e massima per il rilevamento di anomalie e la generazione di allarmi;

`createdBy` / `updatedBy` (`ObjectId`) utenti responsabili della registrazione e modifica del dispositivo;

`createdAt` / `updatedAt` (`Date`) timestamp di creazione e aggiornamento;

2.5.7 Letture sensori

La collezione `sensorreadings` è una *time-series collection* nativa di MongoDB usata per memorizzare lo storico delle misurazioni dei sensori. Il contenuto di questa collezione ha scadenza automatica di 365 giorni (modificabile dalle impostazioni dell'applicazione). Ogni documento contiene:

`_id` (`ObjectId`) identificativo univoco della rilevazione;

`timestamp` (`Date`) istante temporale esatto della misurazione in formato UTC (`timeField`);

`value` (`Number`) valore numerico rilevato;

`unit` (`String`) unità di misura fisica (ad esempio `°C`, `kW`, `m3`);

`metadata` (`Object`) metadati di contesto per il partizionamento efficiente `{sensor (ObjectId), building (ObjectId), sensorType (String)}` (`metaField`);

2.5.8 Allerte

La collezione `alerts` viene usata per memorizzare gli allarmi generati dal sistema a seguito di superamento soglie preimpostate o all'abbassamento dell'efficienza energetica. Ogni documento contiene:

`_id` (`ObjectId`) identificativo univoco dell'allarme;

`building` (`ObjectId`) edificio a cui l'allarme si riferisce (riferimento a `Building`);

`sensor` (`ObjectId`) sensore che ha causato l'anomalia (riferimento a `Sensor`);

`type` (`String`) tipologia dell'anomalia;

`thresholdType` (`String`, `enum` `min|max`) soglia superata (per gli allarmi di soglia);

`severity` (`String`, `enum` `low|medium|high|critical`) livello di gravità calcolato dinamicamente;

`sensorType` (`String`) tipologia del sensore coinvolto (facoltativa);

`location` (`String`) collocazione fisica del sensore al momento dell'evento (facoltativa);

`value` (`Number`) valore riscontrato che ha innescato l'allarme;

`unit` (`String`) unità di misura del valore anomalo;

`limit` (`Number`) soglia di riferimento superata;

`status` (`String`, `enum` `active|acknowledged|resolved`) stato di gestione dell'allarme;

`acknowledgedBy` / `acknowledgedAt` operatore e data di presa in carico;

`resolvedBy` / `resolvedAt` operatore e data di risoluzione;

`createdAt` / `updatedAt` (`Date`) timestamp di creazione e aggiornamento;

2.5.9 ConfigurazioneSistema

La collezione `systemconfigs` è un documento *singleton* gestito tramite il metodo statico `SystemConfig.getOrCreate()`, che memorizza la configurazione globale del sistema. Il documento contiene:

`polling.intervalSeconds` (`Number`) intervallo di aggiornamento automatico per il *frontend* (default 90, range 10–3600);

`polling.autoPollingEnabled` (`Boolean`) abilitazione del *polling* automatico nella dashboard;

`notifications.emailEnabled` (`Boolean`) abilitazione globale delle notifiche di allarme via email;

`database.dataRetentionDays` (`Number`) periodo di conservazione dei delle misurazioni dei sensori (default 365 giorni);

`createdAt` / `updatedAt` (`Date`) timestamp di audit;

2.6 Testing

Le API e le logiche di dominio implementate presentano una test-suite che consente di verificarne il corretto funzionamento. I test sono utilizzati all'interno della configurazione di CI/CD

L'implementazione dei test è organizzata in file `.test.ts` all'interno della directory `apps/api/src/__tests__` (strutturata specularmente al codice sorgente). Durante la fase di testing, tutti i servizi esterni, come login con Google, API Resend e broker MQTT, sono simulati (uso di mock) eccetto per il database. Per simulare il database si è ricorso all'uso della libreria `mongodb-memory-server`, avviando un processo MongoDB effimero per la durata dei test.

La suite conta complessivamente 149 casi di test distribuiti su 18 file, ottenendo oltre il 70% di code coverage.

N.	Descrizione	Test Data	Precondizioni	Dip.	Risultato Atteso	Risultato Ricontrato	Note
1.1	POST /api/v1/auth/session login con credenziali valide	<i>email, password</i>	Utente registrato con password corretta	DB	La richiesta ritorna status 200 mentre nel body è presente token JWT e dati del profilo	La richiesta ritorna status 200 mentre nel body è presente token JWT e dati del profilo	RF1
1.2	POST /api/v1/auth/session login con password errata	<i>email, password</i>	Utente esistente, password non corrispondente	DB	La richiesta ritorna status 401 e come codice d'errore "invalid_credentials"	La richiesta ritorna status 401 e come codice d'errore "invalid_credentials"	RF1
1.3	POST /api/v1/auth/session login con Google ID token	<i>idToken Google</i>	Token valido emesso da Google (GSI) e utente esistente	DB, Google	La richiesta ritorna status 200, utente autenticato e token JWT nel body	La richiesta ritorna status 200, utente autenticato e token JWT nel body	RF1
1.4	POST /api/v1/auth/session login Google utente inesistente	<i>idToken Google</i>	Token valido ma nessuna email registrata	DB, Google	La richiesta ritorna status 404 e codice d'errore <code>oauth_no_account</code>	La richiesta ritorna status 404 e codice d'errore <code>oauth_no_account</code>	RF1
1.5	POST /api/v1/auth/session login Google account disabilitato	<i>idToken Google</i>	Token valido, utente con <code>isDisabled=true</code>	DB, Google	La richiesta ritorna status 403 con errore <code>oauth_account_disabled</code>	La richiesta ritorna status 403 con errore <code>oauth_account_disabled</code>	RF11
1.6	GET /api/v1/auth/session profilo con token valido	<i>JWT token</i>	Token valido, utente attivo	DB	La richiesta ritorna status 200 e i dati del profilo	La richiesta ritorna status 200 e i dati del profilo	RF1
1.7	GET /api/v1/auth/session profilo senza token	-	Nessun token nell'header <code>Authorization</code>	-	La richiesta ritorna status 401 "Unauthorized"	La richiesta ritorna status 401 "Unauthorized"	RF1
1.8	PATCH /api/v1/auth/session modifica profilo	<i>name, language</i>	Utente autenticato	DB	La richiesta ritorna status 200 e il profilo aggiornato mentre status 400 con dati non validi	La richiesta ritorna status 200 e il profilo aggiornato mentre status 400 con dati non validi	RF0
1.9	GET /api/v1/auth/google/config configurazione federata	-	-	-	La risposta ritorna status 200 e il client ID Google configurato	La risposta ritorna status 200 e il client ID Google configurato	RF1
1.10	GET /api/v1/invites?token= validazione invito	<i>token</i>	Invito <code>pending</code> non scaduto	DB	La richiesta ritorna status 200 e dati invito con email	La richiesta ritorna status 200 e dati invito con email	RF11
1.11	PATCH /api/v1/invites/:id setup credenziali locali	<i>token, password, name</i>	Invito valido <code>pending</code>	DB	La richiesta ritorna status 200, l'account attivato e token JWT nel body	La richiesta ritorna status 200, l'account attivato e token JWT nel body	RF11
1.12	PATCH /api/v1/invites/:id accettazione invito tramite Google	<i>idToken Google</i>	Invito valido in attesa di registrazione e nessun account preesistente	DB, Google	La richiesta ritorna status 200, utente collegato e token JWT nel body	La richiesta ritorna status 200, utente collegato e token JWT nel body	RF11

Tabella continuata nella pagina successiva

Tabella continuata dalla pagina precedente

N.	Descrizione	Test Data	Precondizioni	Dip.	Risultato Atteso	Risultato Ricontrato	Note
2.1	GET /api/v1/users elenco gli utenti (admin)	JWT token	Utente admin autenticato	DB	La risposta ritorna status 200 e lista degli utenti priva degli hash della password	La risposta ritorna status 200 e lista degli utenti priva degli hash della password	RF11
2.2	GET /api/v1/users/:id dettaglio utente	id	Utente esistente, ruolo admin	DB	La richiesta ritorna status 200 assieme all'utente con tutti i suoi dettagli	La richiesta ritorna status 200 assieme all'utente con tutti i suoi dettagli	RF11
2.3	GET /api/v1/users/:id utente inesistente	id	ID non presente nel DB	DB	La richiesta ritorna status 404 e il codice d'errore <code>user_not_found</code>	La richiesta ritorna status 404 e il codice d'errore <code>user_not_found</code>	RF11
2.4	PATCH /api/v1/users/:id modifica ruolo	id, role	Utente esistente diverso dall'admin corrente	DB	La richiesta ritorna status 200 e il ruolo aggiornato	La richiesta ritorna status 200 e il ruolo aggiornato	RF11
2.5	PATCH /api/v1/users/:id disabilitazione utente	id, isDisabled	Utente esistente diverso dall'admin corrente	DB	La risposta ritorna status 200 e l'utente disabilitato	La risposta ritorna status 200 e l'utente disabilitato	RF11
2.6	PATCH /api/v1/users/:id modifica del proprio account	id, role	L'id coincide con l'admin richiedente	DB	La richiesta ritorna status 422 con errore <code>cannot_update_own_account</code>	La richiesta ritorna status 422 con errore <code>cannot_update_own_account</code>	RF11
2.7	PATCH /api/v1/users/:id utente inesistente	id	ID non presente nel DB	DB	La richiesta ritorna status 404 e errore <code>user_not_found</code>	La richiesta ritorna status 404 e errore <code>user_not_found</code>	RF11
2.8	PATCH /api/v1/users/:id ruolo non valido	role	Ruolo fuori enum <code>admin operator</code>	DB	La risposta ritorna status 400	La risposta ritorna status 400	RF11
2.9	POST /api/v1/auth/session login utente disabilitato	email, password	Utente con <code>isDisabled=true</code>	DB	La richiesta ritorna status 403	La richiesta ritorna status 403	RF11
2.10	DELETE /api/v1/users/:id cancellazione utente	id	Utente esistente diverso dall'admin corrente	DB	La richiesta ritorna soltanto con status 204	La richiesta ritorna soltanto con status 204	RF11
2.11	DELETE /api/v1/users/:id cancellazione di sé stesso	id	L'id coincide con l'admin corrente	DB	La richiesta ritorna con status 422 con codice d'errore <code>cannot_delete_own_account</code>	La richiesta ritorna con status 422 con codice d'errore <code>cannot_delete_own_account</code>	RF11
2.12	DELETE /api/v1/users/:id utente inesistente	id	ID non presente nel DB	DB	La richiesta ritorna con status 404 e codice di errore <code>user_not_found</code>	La richiesta ritorna con status 404 e codice di errore <code>user_not_found</code>	RF11
2.13	POST /api/v1/invites creazione invito	email, role	Email non registrata, nessun invito pendente	DB, Resend	La richiesta ritorna con status 201, creando con successo l'invito e inviando la mail	La richiesta ritorna con status 201, creando con successo l'invito e inviando la mail	RF11
2.14	POST /api/v1/invites email già registrata	email, role	Email di utente esistente	DB	La richiesta ritorna status 409 e errore <code>user_email_exists</code>	La richiesta ritorna status 409 e errore <code>user_email_exists</code>	RF11
2.15	POST /api/v1/invites invito pendente esistente	email, role	Invito <code>pending</code> per la stessa email	DB	La richiesta ritorna status 409 con codice d'errore <code>invite_pending_exists</code>	La richiesta ritorna status 409 con codice d'errore <code>invite_pending_exists</code>	RF11
2.16	POST /api/v1/invites email non valida	email	Formato email errato	DB	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF11, RNF5
2.17	POST /api/v1/invites ruolo non valido	role	Ruolo fuori enum	DB	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF11, RNF5

Tabella continuata nella pagina successiva

Tabella continuata dalla pagina precedente

N.	Descrizione	Test Data	Precondizioni	Dip.	Risultato Atteso	Risultato Ricontrato	Note
2.18	POST /api/v1/invites email mancante	-	Campo <i>email</i> assente	DB	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF11, RNF5
2.19	POST /api/v1/invites ruolo mancante	-	Campo <i>role</i> assente	DB	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF11, RNF5
2.20	GET /api/v1/invites elenco inviti	-	Ruolo <i>admin</i>	DB	La richiesta ritorna status 200 e l'elenco completo degli inviti nel body	La richiesta ritorna status 200 e l'elenco completo degli inviti nel body	RF11
2.21	GET /api/v1/invites/:id dettaglio invito	<i>id</i>	Invito esistente, ruolo <i>admin</i>	DB	La richiesta ritorna status 200, e nel body i dettagli dell'invito	La richiesta ritorna status 200, e nel body i dettagli dell'invito	RF11
2.22	GET /api/v1/invites/:id invito inesistente	<i>id</i>	ID non presente nel DB	DB	La richiesta ritorna status 404 e codice d'errore <i>invite_not_found</i>	La richiesta ritorna status 404 e codice d'errore <i>invite_not_found</i>	RF11
2.23	DELETE /api/v1/invites/:id revoca invito già revocato	<i>id</i>	Invito in stato <i>revoked</i>	DB	Errore 409 transizione vietata	Errore 409 transizione vietata	RF11
2.24	DELETE /api/v1/invites/:id revoca invito pendente	<i>id</i>	Invito in stato <i>pending</i>	DB	La richiesta ritorna lo status 200, stato <i>revoked</i>	La richiesta ritorna lo status 200, stato <i>revoked</i>	RF11
2.25	DELETE /api/v1/invites/:id invito inesistente	<i>id</i>	ID non presente nel DB	DB	Errore 404 invito non trovato	Errore 404 invito non trovato	RF11
2.26	GET /api/v1/invites?token=lookup token valido	<i>token</i>	Token grezzo corrispondente a invito <i>pending</i>	DB	La richiesta ritorna lo status 200, dati invito	La richiesta ritorna lo status 200, dati invito	RF11
2.27	GET /api/v1/invites?token=token sconosciuto	<i>token</i>	Nessun invito corrispondente	DB	Errore 404 token non valido	Errore 404 token non valido	RF11
2.28	PATCH /api/v1/invites/:id token di altro invito	<i>token</i>	Token legato a invito diverso	DB	Errore 404 token non valido	Errore 404 token non valido	RF11
3.1	GET /api/v1/building-types lista tipologie	-	Ruolo <i>admin</i>	DB	La richiesta ritorna lo status 200, array tipologie	La richiesta ritorna lo status 200, array tipologie	RF4
3.2	GET /api/v1/building-types/:id tipologia inesistente	<i>id</i>	ID non presente nel DB	DB	Errore 404 non trovata	Errore 404 non trovata	RF5
3.3	GET /api/v1/building-types/:id ObjectId malformato	<i>id</i>	ID non valido	-	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF5, RNF5
3.4	POST /api/v1/building-types creazione tipologia	<i>name, desc.</i>	Ruolo <i>admin</i> , nome univoco	DB	La richiesta ritorna lo status 201, tipologia creata	La richiesta ritorna lo status 201, tipologia creata	RF5
3.5	POST /api/v1/building-types nome duplicato	<i>name</i>	Nome già esistente	DB	Errore 409 violazione unicità	Errore 409 violazione unicità	RF5
3.6	POST /api/v1/building-types dati non validi	<i>name</i>	Nome vuoto o assente	DB	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF5, RNF5
3.7	PATCH /api/v1/building-types/:id modifica tipologia	<i>id, name</i>	Tipologia esistente, ruolo <i>admin</i>	DB	La richiesta ritorna lo status 200, tipologia aggiornata	La richiesta ritorna lo status 200, tipologia aggiornata	RF5
3.8	PATCH /api/v1/building-types/:id nome duplicato	<i>name</i>	Nome già in uso	DB	Errore 409 violazione unicità	Errore 409 violazione unicità	RF5

Tabella continuata nella pagina successiva

Tabella continuata dalla pagina precedente

N.	Descrizione	Test Data	Precondizioni	Dip.	Risultato Atteso	Risultato Ricontrato	Note
3.9	PATCH /api/v1/building-types/:id tipologia inesistente	id	ID non presente nel DB	DB	Errore 404 non trovata	Errore 404 non trovata	RF5
3.10	DELETE /api/v1/building-types/:id cancellazione tipologia	id	Tipologia esistente non referenziata	DB	La richiesta ritorna lo status 200, tipologia rimossa	La richiesta ritorna lo status 200, tipologia rimossa	RF5
3.11	DELETE /api/v1/building-types/:id tipologia in uso	id	Referenziata da almeno un edificio	DB	Errore 400 vincolo referenziale	Errore 400 vincolo referenziale	RF5
3.12	DELETE /api/v1/building-types/:id tipologia inesistente	id	ID non presente nel DB	DB	Errore 404 non trovata	Errore 404 non trovata	RF5
4.1	POST /api/v1/buildings creazione edificio	name, addr, surf, loc, type	Dati validi, GeoJSON valido, tipologia esistente, ruolo admin	DB	La richiesta ritorna lo status 201, edificio creato	La richiesta ritorna lo status 201, edificio creato	RF5
4.2	POST /api/v1/buildings creazione con ruolo operator	name, addr, surf, loc, type	Dati validi, ruolo operator	DB	La richiesta ritorna lo status 201, edificio creato	La richiesta ritorna lo status 201, edificio creato	RF5
4.3	POST /api/v1/buildings dati non validi	surface neg.	Campi fuori range o mancanti	-	La richiesta ritorna con status 400 dovuto ad un errore di validazione Zod	La richiesta ritorna con status 400 dovuto ad un errore di validazione Zod	RF5, RNF5
4.4	POST /api/v1/buildings tipologia inesistente	buildingType	Riferimento tipologia assente	DB	Errore 404 tipologia non trovata	Errore 404 tipologia non trovata	RF5
4.5	PATCH /api/v1/buildings/:id modifica edificio	id, name	Edificio esistente	DB	La richiesta ritorna lo status 200, documento aggiornato	La richiesta ritorna lo status 200, documento aggiornato	RF5
4.6	GET /api/v1/buildings/:id edificio inesistente	id	ID non presente nel DB	DB	Errore 404 non trovato	Errore 404 non trovato	RF5
4.7	PATCH /api/v1/buildings/:id soglie COP valide	efficiencyThreshold	Edificio esistente, minCOP presente	DB	La richiesta ritorna lo status 200, soglie salvate	La richiesta ritorna lo status 200, soglie salvate	RF6
4.8	PATCH /api/v1/buildings/:id soglia senza minCOP	efficiencyThreshold	Edificio senza minCOP	DB	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF6, RNF5
4.9	PATCH /api/v1/buildings/:id soglie su teleriscaldamento	efficiencyThreshold	Edificio a teleriscaldamento	DB	Errore 400 non applicabile	Errore 400 non applicabile	RF6
4.10	DELETE /api/v1/buildings/:id cancellazione a cascata	id	Edificio con sensori, allarmi e letture	DB	La richiesta ritorna lo status 200, entità collegate rimosse	La richiesta ritorna lo status 200, entità collegate rimosse	RF5
4.11	GET /api/v1/buildings ricerca per nome	name	Edifici presenti nel DB	DB	La richiesta ritorna lo status 200, lista filtrata	La richiesta ritorna lo status 200, lista filtrata	RF3
4.12	GET /api/v1/buildings ricerca per indirizzo	address	Edifici presenti nel DB	DB	La richiesta ritorna lo status 200, lista filtrata	La richiesta ritorna lo status 200, lista filtrata	RF3
4.13	GET /api/v1/buildings filtro per zona	zone	Edifici presenti nel DB	DB	La richiesta ritorna lo status 200, lista filtrata	La richiesta ritorna lo status 200, lista filtrata	RF3
4.14	GET /api/v1/buildings filtro per tipologia	type	Edifici presenti nel DB	DB	La richiesta ritorna lo status 200, lista filtrata	La richiesta ritorna lo status 200, lista filtrata	RF3
4.15	GET /api/v1/buildings filtri combinati	name, zone, status	Edifici presenti nel DB	DB	La richiesta ritorna lo status 200, lista filtrata	La richiesta ritorna lo status 200, lista filtrata	RF3

Tabella continuata nella pagina successiva

Tabella continuata dalla pagina precedente

N.	Descrizione	Test Data	Precondizioni	Dip.	Risultato Atteso	Risultato Ricontrato	Note
4.16	GET /api/v1/buildings campi obbligatori in elenco	-	Edifici presenti nel DB	DB	La richiesta ritorna lo status 200, nome, indirizzo, superficie, impianto, stato, aggiornamento	La richiesta ritorna lo status 200, nome, indirizzo, superficie, impianto, stato, aggiornamento	RF3
4.17	GET /api/v1/buildings/:id dettaglio edificio	id	Edificio esistente	DB	La richiesta ritorna lo status 200, anagrafica completa	La richiesta ritorna lo status 200, anagrafica completa	RF4
4.18	GET /api/v1/buildings/:id serie storica	startDate, endDate, history	Lecture nell'intervallo	DB	La richiesta ritorna lo status 200, rilevazioni ordinate	La richiesta ritorna lo status 200, rilevazioni ordinate	RF4, RF8
4.19	GET /api/v1/buildings/:id filtro per data	startDate, endDate, history	Lecture nell'intervallo	DB	La richiesta ritorna lo status 200, solo rilevazioni nel periodo	La richiesta ritorna lo status 200, solo rilevazioni nel periodo	RF4, RF8
4.20	GET /api/v1/buildings/:id tipi di intervallo	interval, history	Lecture presenti	DB	La richiesta ritorna lo status 200, aggregazione per intervallo	La richiesta ritorna lo status 200, aggregazione per intervallo	RF4
4.21	GET /api/v1/buildings/:id edificio inesistente	id	ID non presente nel DB	DB	Errore 404 non trovato	Errore 404 non trovato	RF12
4.22	GET /api/v1/buildings/:id data non ISO	startDate, efficiency	Data malformata	-	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF12, RNF5
4.23	GET /api/v1/buildings/:id senza dati	id, date, efficiency	Nessuna lettura nel periodo	DB	La richiesta ritorna lo status 200, metriche a null	La richiesta ritorna lo status 200, metriche a null	RF12
4.24	GET /api/v1/buildings/:id calcolo COP	id, date, efficiency	Consumi e temperature presenti	DB	La richiesta ritorna lo status 200, COP e dispersione calcolati	La richiesta ritorna lo status 200, COP e dispersione calcolati	RF12
4.25	GET /api/v1/buildings/:id temperature meteo	id, date, efficiency	Senza sonda esterna, API meteo disponibile	DB, meteo	La richiesta ritorna lo status 200, COP e H da fallback meteo	La richiesta ritorna lo status 200, COP e H da fallback meteo	RNF8 – Fonte meteo
4.26	GET /api/v1/buildings/:id consumi gas	id, date, efficiency	Lecture gas cumulative	DB	La richiesta ritorna lo status 200, consumo da m ³ × PCI	La richiesta ritorna lo status 200, consumo da m ³ × PCI	RF12
4.27	GET /api/v1/buildings/:id teleriscaldamento	id, date, efficiency	Impianto a teleriscaldamento	DB	La richiesta ritorna lo status 200, COP a null	La richiesta ritorna lo status 200, COP a null	RF6, RF12
4.28	GET /api/v1/buildings/:id calcolo efficienza	id, date, efficiency	Consumi e temperature presenti	DB	La richiesta ritorna lo status 200, metriche di efficienza	La richiesta ritorna lo status 200, metriche di efficienza	RF12
4.29	GET /api/v1/buildings/:id date non valide	startDate, endDate, efficiency	Intervallo malformato	-	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF12, RNF5
5.1	POST /api/v1/sensors registrazione sensore	building, sensorType, location	Edificio esistente, ruolo admin	DB	La richiesta ritorna lo status 201, sensore active	La richiesta ritorna lo status 201, sensore active	RF5
5.2	POST /api/v1/sensors registrazione con operator	building, sensorType, location	Edificio esistente, ruolo operator	DB	La richiesta ritorna lo status 201, sensore creato	La richiesta ritorna lo status 201, sensore creato	RF5
5.3	POST /api/v1/sensors matricola duplicata	serialNumber	Matricola già registrata	DB	Errore 400 violazione unicità	Errore 400 violazione unicità	RF5
5.4	POST /api/v1/sensors dati non validi	sensorType, location	Tipo fuori enum o posizione vuota	DB	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF5, RNF5

Tabella continuata nella pagina successiva

Tabella continuata dalla pagina precedente

N.	Descrizione	Test Data	Precondizioni	Dip.	Risultato Atteso	Risultato Ricontrato	Note
5.5	GET /api/v1/sensors/:id dettaglio sensore	id	Sensore esistente	DB	La richiesta ritorna lo status 200, dettaglio completo	La richiesta ritorna lo status 200, dettaglio completo	RF7
5.6	GET /api/v1/sensors/:id sensore inesistente	id	ID non presente nel DB	DB	Errore 404 non trovato	Errore 404 non trovato	RF7
5.7	PATCH /api/v1/sensors/:id soglie min/max	id, <i>minThreshold</i> , <i>maxThreshold</i>	Sensore esistente	DB	La richiesta ritorna lo status 200, soglie aggiornate	La richiesta ritorna lo status 200, soglie aggiornate	RF9
5.8	PATCH /api/v1/sensors/:id sensore inesistente	id	ID non presente nel DB	DB	Errore 404 non trovato	Errore 404 non trovato	RF9
5.9	DELETE /api/v1/sensors/:id cancellazione a cascata	id	Sensore con letture e allarmi	DB	La richiesta ritorna lo status 200, dati associati rimossi	La richiesta ritorna lo status 200, dati associati rimossi	RF5
5.10	DELETE /api/v1/sensors/:id sensore inesistente	id	ID non presente nel DB	DB	Errore 404 non trovato	Errore 404 non trovato	RF5
5.11	Controllo inattività sensore oltre 2× intervallo	id	Nessuna lettura da oltre il doppio dell'intervallo	DB	Stato inactive , isOffline=true	Stato inactive , isOffline=true	RF7, RNF4
5.12	Controllo attività sensore entro 2× intervallo	id	Lettura recente entro il doppio dell'intervallo	DB	Stato active mantenuto	Stato active mantenuto	RF7
5.13	Sensori in manutenzione o errore con letture vecchie	id	Stato maintenance o error	DB	Stato invariato	Stato invariato	RF7
5.14	Sensore attivo senza alcuna lettura	id	lastReading assente	DB	Stato inactive	Stato inactive	RF7, RNF4
5.15	GET /api/v1/sensors/:id/readings storico letture	id, date	Letture presenti	DB	La richiesta ritorna lo status 200, serie storica	La richiesta ritorna lo status 200, serie storica	RF8
5.16	GET /api/v1/sensors/:id/readings sensore inesistente	id	ID non presente nel DB	DB	Errore 404 non trovato	Errore 404 non trovato	RF8
6.1	GET /api/v1/alerts elenco allarmi	<i>status</i> , <i>severity</i> , <i>buildingId</i>	Allarmi presenti nel DB	DB	La richiesta ritorna lo status 200, elenco per gravità/data	La richiesta ritorna lo status 200, elenco per gravità/data	RF6
6.2	PATCH /api/v1/alerts/:id presa in carico	id	Allarme in stato active	DB	La richiesta ritorna lo status 200, operatore e timestamp registrati	La richiesta ritorna lo status 200, operatore e timestamp registrati	RF6
6.3	PATCH /api/v1/alerts/:id presa in carico non valida	id	Allarme già gestito o risolto	DB	Errore 409 transizione vietata	Errore 409 transizione vietata	RF6
6.4	PATCH /api/v1/alerts/:id risoluzione allarme	id	Allarme non risolto	DB	La richiesta ritorna lo status 200, allarme resolved	La richiesta ritorna lo status 200, allarme resolved	RF6
6.5	GET /api/v1/alerts/:id dettaglio allarme	id	Allarme esistente	DB	La richiesta ritorna lo status 200, edificio, valori e link	La richiesta ritorna lo status 200, edificio, valori e link	RF6
6.6	GET /api/v1/alerts/:id allarme inesistente	id	ID non presente nel DB	DB	Errore 404 non trovato	Errore 404 non trovato	RF6
7.1	GET /api/v1/readings intervallo date invalido	<i>startDate</i> , <i>endDate</i>	Date mancanti o invertite	DB	Errore 400/422	Errore 400/422	RF10, RNF5
7.2	GET /api/v1/readings esportazione CSV	<i>buildingIds</i> , date	Letture nell'intervallo	DB	La richiesta ritorna lo status 200, file text/csv	La richiesta ritorna lo status 200, file text/csv	RF10
7.3	GET /api/v1/readings edificio inesistente	<i>buildingIds</i>	ID non presente nel DB	DB	Errore 404 non trovato	Errore 404 non trovato	RF10

Tabella continuata nella pagina successiva

Tabella continuata dalla pagina precedente

N.	Descrizione	Test Data	Precondizioni	Dip.	Risultato Atteso	Risultato Ricontrato	Note
7.4	GET /api/v1/reports intervallo date invalido	<i>startDate, endDate</i>	Date mancanti o invertite	DB	Errore 400/422	Errore 400/422	RF10, RNF5
7.5	GET /api/v1/reports formato non valido	<i>format</i>	Formato fuori enum	-	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF10, RNF5
7.6	GET /api/v1/reports edificio inesistente	<i>buildingIds</i>	ID non presente nel DB	DB	Errore 404 non trovato	Errore 404 non trovato	RF10
7.7	GET /api/v1/reports report PDF	<i>buildingIds, date, pdf</i>	Edifici e letture presenti	DB	La richiesta ritorna lo status 200, application/pdf	La richiesta ritorna lo status 200, application/pdf	RF10
7.8	GET /api/v1/reports report Excel	<i>buildingIds, date, xlsx</i>	Edifici e letture presenti	DB	La richiesta ritorna lo status 200, foglio OpenXML	La richiesta ritorna lo status 200, foglio OpenXML	RF10
7.9	GET /api/v1/reports etichette localizzate	<i>lang</i>	Lingua richiesta valida	DB	La richiesta ritorna lo status 200, report nella lingua	La richiesta ritorna lo status 200, report nella lingua	RF0
7.10	GET /api/health stato servizio	-	Nessuna	-	La richiesta ritorna lo status 200, {status: "ok"}	La richiesta ritorna lo status 200, {status: "ok"}	RNF4
7.11	GET /api/v1/metrics contatori aggregati	-	Sensori e allarmi presenti	DB	La richiesta ritorna lo status 200, totali, attivi e consumi	La richiesta ritorna lo status 200, totali, attivi e consumi	RF2
7.12	GET /api/v1/metrics/timeseries serie aggregate	<i>startDate, endDate, interval</i>	Intervallo valido	DB	La richiesta ritorna lo status 200, serie per periodo	La richiesta ritorna lo status 200, serie per periodo	RF2
7.13	GET /api/v1/metrics/timeseries intervallo invalido	<i>startDate, endDate</i>	Date invertite	-	Errore 422 range invalido	Errore 422 range invalido	RF2, RNF5 – Convalida
7.14	GET /api/settings configurazione sistema	-	Ruolo admin	DB	La richiesta ritorna lo status 200, configurazione globale	La richiesta ritorna lo status 200, configurazione globale	RNF4
7.15	PATCH /api/settings retention e collMod	<i>dataRetentionDays</i>	Valore entro i limiti	DB	La richiesta ritorna lo status 200, retention e TTL aggiornati	La richiesta ritorna lo status 200, retention e TTL aggiornati	RNF4
8.1	Ingestione lettura e lastReading	<i>sensorId, value</i>	Sensore esistente	DB	Lettura persistita, sensore aggiornato	Lettura persistita, sensore aggiornato	RF8
8.2	Riattivazione sensore inattivo alla nuova lettura	<i>sensorId</i>	Sensore inactive	DB	Stato active ripristinato	Stato active ripristinato	RF7, RNF4
8.3	Lettura entro le soglie senza allarmi	<i>value</i>	Valore tra min e max	DB	Nessun allarme creato	Nessun allarme creato	RF9
8.4	Superamento soglia massima	<i>value > max</i>	Soglia configurata	DB	Allarme max attivo creato	Allarme max attivo creato	RF6
8.5	Superamento soglia minima	<i>value < min</i>	Soglia configurata	DB	Allarme min attivo creato	Allarme min attivo creato	RF6
8.6	Lettura per sensore inesistente	<i>sensorId</i>	Sensore assente nel DB	DB	Errore SensorNotFound	Errore SensorNotFound	RF8
8.7	Allarme COP sotto soglia	COP < minCOP	Soglie efficienza abilitate	DB	Allarme efficiency_cop_low creato	Allarme efficiency_cop_low creato	RF6
8.8	Deduplicazione allarmi COP attivi	COP < minCOP	Allarme attivo esistente	DB	Nessun duplicato creato	Nessun duplicato creato	RF6
8.9	Risoluzione allarme COP al recupero	COP > minCOP	Allarme attivo esistente	DB	Allarme risolto	Allarme risolto	RF6
8.10	Email di allarme multilingua agli attivi	Allarme creato	Utenti attivi, email abilitate	DB, Resend	Email nella lingua di ognuno, disabilitati esclusi	Email nella lingua di ognuno, disabilitati esclusi	RF9, RF0
8.11	Nessuna email con notifiche disabilitate	Allarme creato	Toggle email spento	DB	Nessun invio	Nessun invio	RF9
8.12	Destinatario fallito non blocca gli altri	Allarme creato	Un invio fallisce	DB, Resend	Gli altri ricevono l'email	Gli altri ricevono l'email	RF9

Tabella continuata nella pagina successiva

Tabella continuata dalla pagina precedente

N.	Descrizione	Test Data	Precondizioni	Dip.	Risultato Atteso	Risultato Riscontrato	Note
8.13	Simulatore emette una lettura per sensore	Sensori registrati	Simulatore avviato	DB	Lecture valide per ogni sensore	Lecture valide per ogni sensore	RF8
8.14	Simulatore emette consumi kW pompa di calore	Pompa di calore accesa	Simulatore avviato	DB	Lettura energy_meter in kW	Lettura energy_meter in kW	RF8
8.15	Simulatore emette contatore gas con resume	Caldaia a gas	Lettura precedente presente	DB	Lettura gas_meter in m ³ dal valore salvato	Lettura gas_meter in m ³ dal valore salvato	RF12
8.16	Simulatore incrementa odometro gas	Caldaia accesa	Simulatore avviato	DB	Contatore cumulativo crescente	Contatore cumulativo crescente	RF12 – Consumi
8.17	Messaggio MQTT valido su sensors/+/readings	<i>topic, value, unit</i>	Sensore esistente, payload conforme	DB, MQTT	Lettura delegata al servizio	Lettura delegata al servizio	RF8
8.18	Messaggio MQTT con topic non valido	<i>topic</i>	Topic senza ID sensore	MQTT	Messaggio ignorato senza errori	Messaggio ignorato senza errori	RNF4
8.19	Messaggio MQTT con JSON malformato	<i>payload</i>	Payload corrotto	MQTT	Messaggio scartato senza crash	Messaggio scartato senza crash	RNF4
8.20	Messaggio MQTT con payload non valido	<i>value</i>	Validazione fallita	MQTT	Messaggio scartato senza crash	Messaggio scartato senza crash	RNF5
9.1	Formati email validi accettati	<i>email</i>	Formati conformi	-	Nessun errore di formato	Nessun errore di formato	RNF5
9.2	Formati email non validi rifiutati	<i>email</i>	Formati errati	-	Errore 400 formato	Errore 400 formato	RNF5
9.3	JSON malformato rifiutato	<i>body</i>	Corpo non JSON	-	Errore 400 parsing	Errore 400 parsing	RNF5
9.4	Corpo richiesta vuoto rifiutato	-	Corpo assente	-	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RNF5
9.5	Tipi di campo errati rifiutati	<i>email, password</i>	Tipi non stringa	-	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RNF5
9.6	Campo email mancante rifiutato	-	<i>email</i> assente	-	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RNF5
9.7	Parametri token invito validi	<i>token</i>	Token ben formato	DB	Lookup eseguita	Lookup eseguita	RF11
9.8	Token invito invalido rifiutato	<i>token</i>	Token inesistente	DB	Errore 404/422	Errore 404/422	RF11
9.9	Ruoli validi accettati	<i>role</i>	admin operator	DB	Nessun errore di ruolo	Nessun errore di ruolo	RF11
9.10	Ruoli non validi rifiutati	<i>role</i>	Ruolo fuori enum	-	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF11, RNF5
9.11	Campo ruolo mancante rifiutato	-	<i>role</i> assente	-	La richiesta ritorna con status 400 dovuto ad un errore di validazione	La richiesta ritorna con status 400 dovuto ad un errore di validazione	RF11, RNF5

3 *FrontEnd*

Di seguito viene illustrata la parte *FrontEnd* del progetto.

3.1 Autenticazione

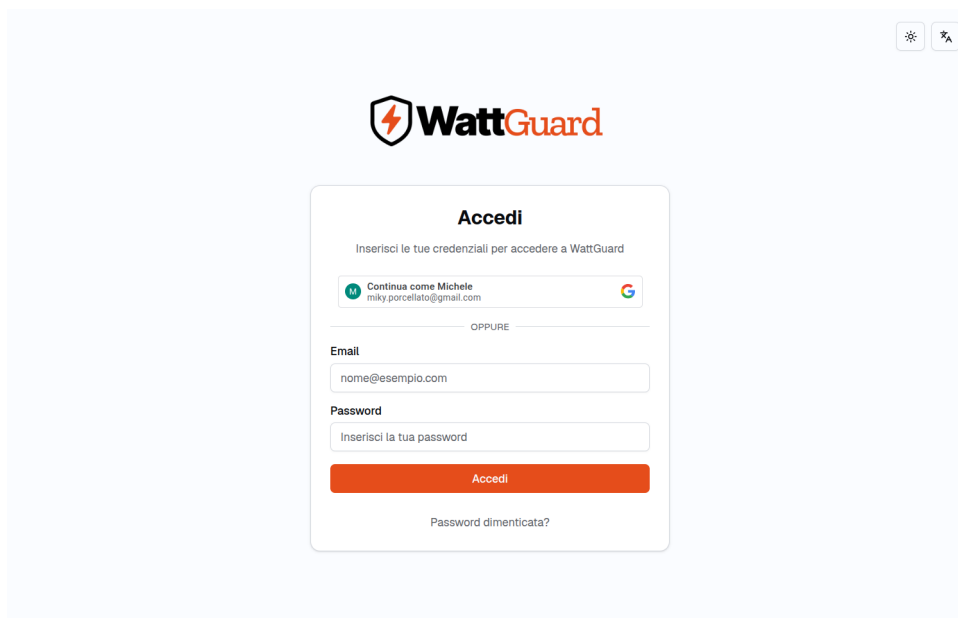


Figura 1: Pagina di login

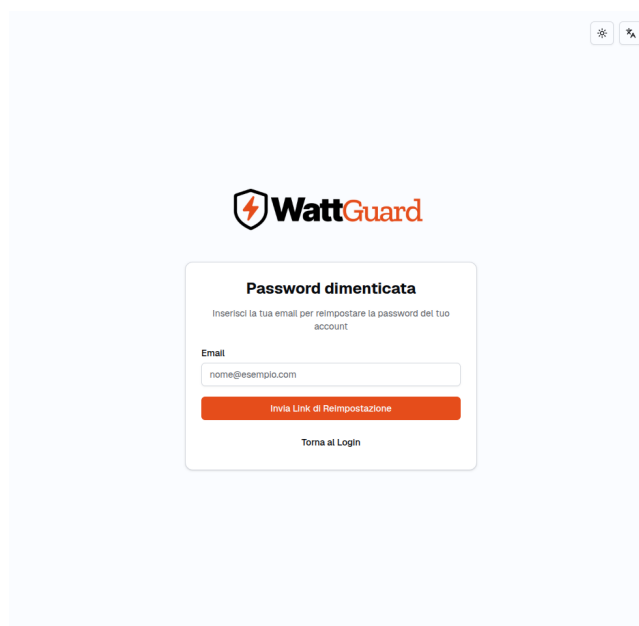


Figura 2: Reset Password

Nella fig. 1 è presente la pagina principale di login, alla quale l'utente è condotto se non è attualmente loggato o il token di autenticazione JWT è scaduto. È possibile autenticarsi sia tramite credenziali locali, inserendo e-mail e password, sia attraverso il pulsante "Accedi con Google"

Durante la risoluzione dello stato di autenticazione viene mostrato uno *spinner*. Se l'utente viene autenticato con successo, viene reindirizzato alla dashboard principale.

Nella stessa sezione è possibile procedere con il recupero della password. (fig. 2)

3.2 Accettazione Invito

The figure displays two sequential screenshots of the 'Accetta Invito' (Accept Invitation) form in the WattGuard application.

Left Screenshot: The form is titled 'Accetta Invito' with the subtitle 'Sei stato invitato a unirti a WattGuard'. It shows a pre-filled email 'michele.porcellato@studenti.unith.it' and a role 'Operatore'. Below this, there is a 'Continua con Google' button with the Google logo and a 'Crea account con password' button. A separator line with the word 'OPPURE' (or) is positioned between the two options.

Right Screenshot: This screenshot shows the form after the user has chosen to create an account with a password. The 'Email' and 'Ruolo' fields remain pre-filled. The 'Nome Completo' (Full Name) field is now active, with 'Mario Rossi' entered. Below it, a note states 'Il tuo nome come verrà visualizzato nel sistema.' (Your name as it will be displayed in the system). The 'Password' field is also active, with the placeholder 'Almeno 8 caratteri' (At least 8 characters). A note below it says 'Scegli una password sicura di almeno 8 caratteri.' (Choose a secure password of at least 8 characters). The 'Conferma Password' (Confirm Password) field is active with the placeholder 'Ripeti la password' (Repeat the password). At the bottom, there are 'Indietro' (Back) and 'Crea Account' buttons.

Figura 3: Flow di accettazione invito

Come illustrato nella fig. 3 si illustrano i passaggi necessari per accettare l'invito e completare la registrazione. Durante la registrazione è possibile procedere con il login con Google oppure creare manualmente l'utente. Se si decide di procedere con la creazione manuale, è richiesto compilare il modulo con nome utente e password.

3.3 Layout della dashboard

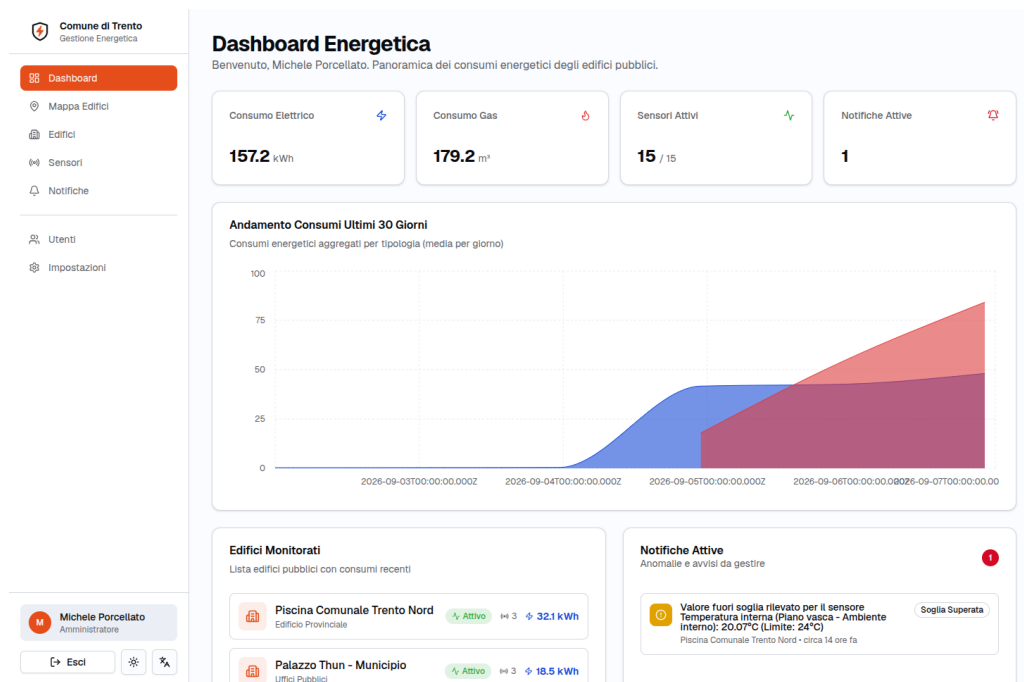


Figura 4: Dashboard principale

Dopo l'autenticazione l'utente si trova all'interno della dashboard. (fig. 4).

Osserviamo la presenza della nav-bar a lato, la quale indica che pagina è attualmente selezionata (dashboard generare, mappa degli edifici, ricerca edifici, sensori e notifiche) e informazioni sull'utente attualmente loggato. In basso si trova il pulsante per eseguire il logout, il selettore per cambiare il tema predefinito e il selettore per cambiare la lingua del frontend (it/en/de). Le sezioni di utenti e impostazioni sono presenti soltanto per gli amministratori.

La pagina della dashboard principale contiene i seguenti componenti:

- **Informazioni complessive del sistema:** Questa sezione contiene il consumo elettrico complessivo attuale, il consumo di gas degli ultimi 30 giorni, il sensori attivi (non in manutenzione o offline) e le notifiche presenti.
- **Grafico andamento dei consumi:** Il grafico mostra i consumi energetici aggregati per tipologia (celeste il consumo elettrico mentre il gas in rosso) degli ultimi 30 giorni. Facendo hover sul grafico compare un tooltip che mostra i dettagli del singolo punto selezionato.
- **Edifici Monitorati:** Mostra la lista degli edifici con i loro consumi recenti. È presente inoltre l'indicatore di quanti sensori sono presenti nell'edificio, se l'edificio è attivo o dismesso e il loro consumo istantaneo di corrente elettrica.
- **Notifiche attive:** Questa sezione serve per ricordare agli amministratori e agli operatori di eventuali anomalie presenti negli edifici monitorati.

I dati presenti nella dashboard vengono aggiornati automaticamente ogni 90 secondi. Questo intervallo è configurabile all'interno delle impostazioni di sistema.

3.4 Mappa degli edifici

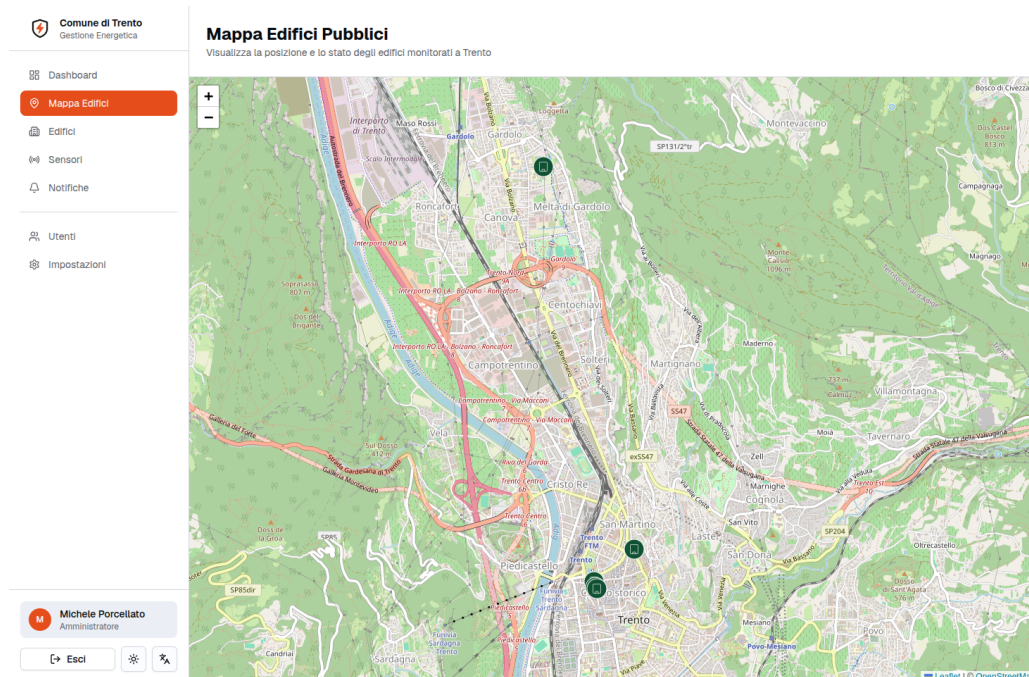


Figura 5: Mappa edifici

Nella fig. 5 si mostra la pagina contenente la disposizione geografica degli edifici monitorati su una mappa interattiva. Ogni edificio è rappresentato da un marcatore; selezionandolo, l'utente accede al dettaglio dell'edificio. Il marcatore è verde se l'edificio risulta attivo, grigio se dismesso.

3.5 Edifici

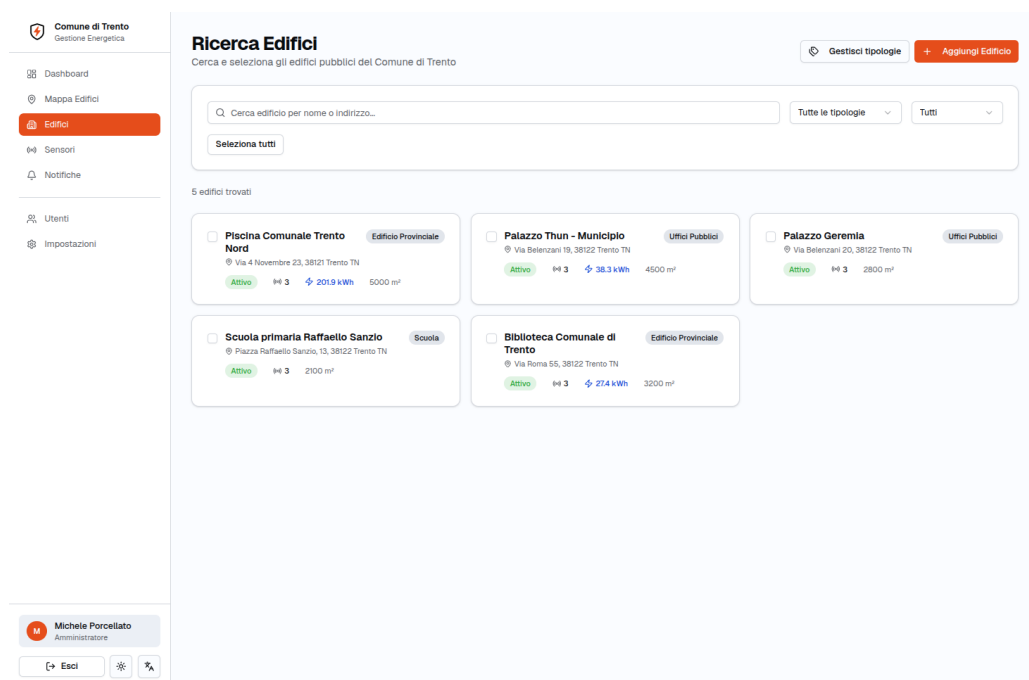


Figura 6: Elenco edifici con ricerca

Aggiungi Nuovo Edificio

Compila i campi per registrare un nuovo edificio nel sistema.

Nome *

Indirizzo *

Inserisci l'indirizzo e premi il pulsante di ricerca per trovare le coordinate automaticamente

Superficie (m²) * **Altezza Soffitto (m) ***

Anno di costruzione

Tipo edificio * + Nuova tipologia

Seleziona tipo edificio ▾

Tipo riscaldamento * **Zona geografica ***

Posizione *

Lattitudine Longitudine

Clicca sulla mappa per posizionare il marker oppure inserisci le coordinate manualmente

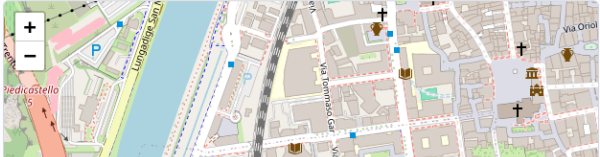


Figura 7: Dialogo per la creazione di un nuovo edificio

In fig. 6 è presente la sezione dedicata alla visualizzazione di tutti gli edifici. In alto è presente una barra di ricerca che permette di ricercare gli edifici per nome, indirizzo, zona geografica, tipologia e stato operativo. Ogni edificio elencato presenta il nome, l'indirizzo, il numero di sensori attivi e il consumo di corrente dell'edificio. È inoltre possibile registrare nuovi edifici, come si vede in fig. 7, inserendo i dati anagrafici dell'edificio e la posizione geografica (si accettato sia coordinate che indirizzo).

3.5.1 Confronto Edifici

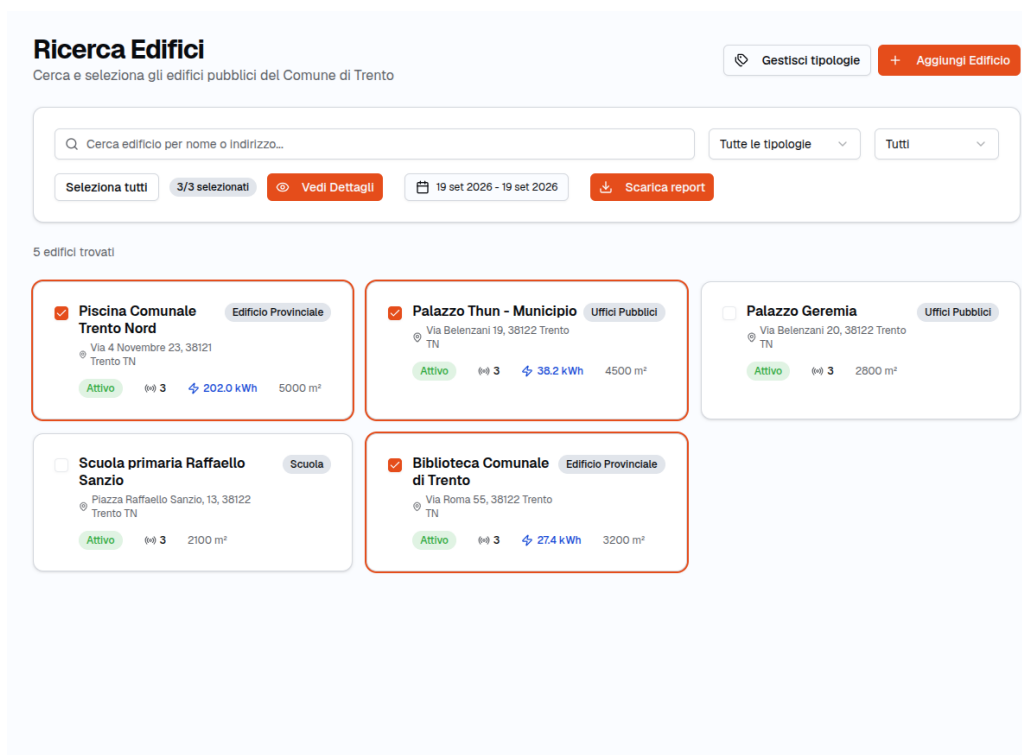


Figura 8: Selezione di più edifici

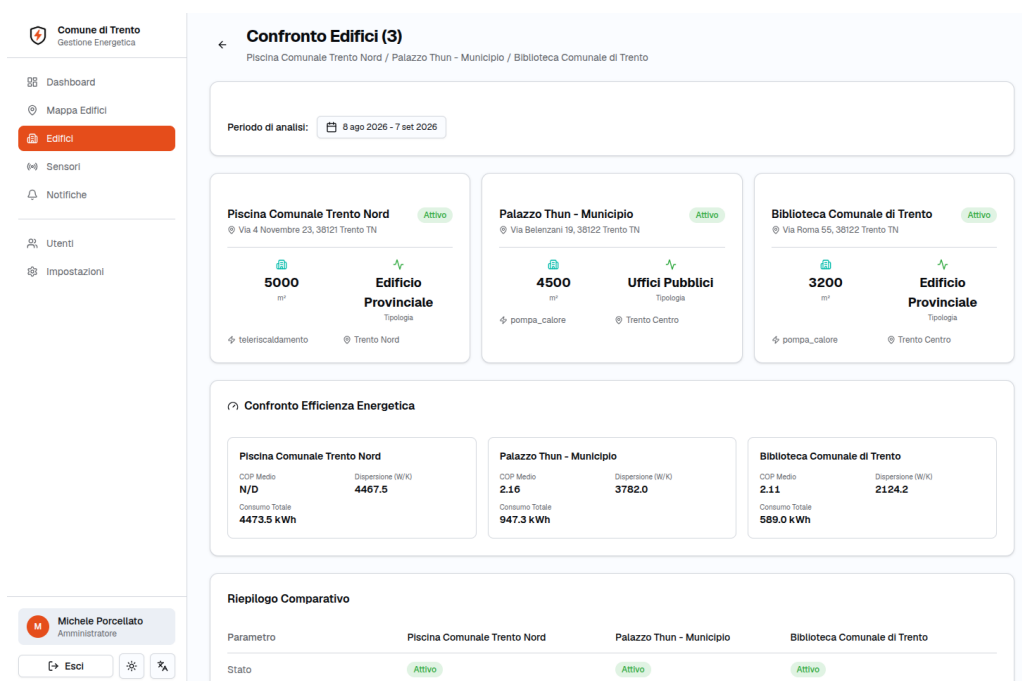


Figura 9: Pagina di comparazione

È possibile confrontare più edifici, selezionandone al massimo 3, come si vede in fig. 8. Da qui è possibile scaricare sia il report dell'efficienza dei vari edifici in PDF che i valori dei vari sensori in CSV. Cliccando su "Vedi dettagli" si apre la visualizzazione comparativa (fig. 9). Questa pagina mostra intuitivamente l'efficienza dei vari edifici e lo stato generale degli stessi.

3.6 Dettaglio edificio

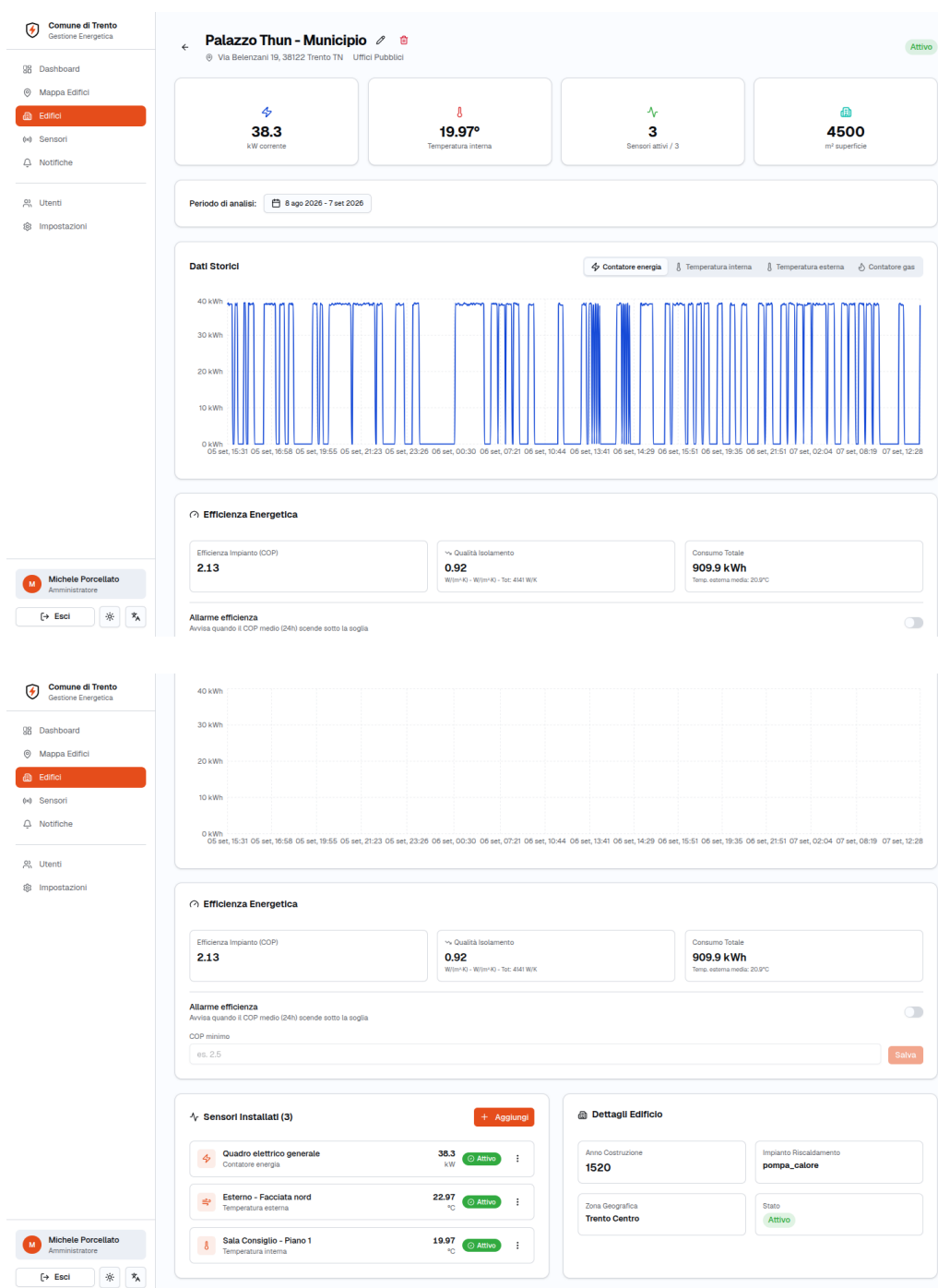


Figura 10: Pagina di dettaglio dell'edificio

La pagina di dettaglio di un edificio riunisce i dati anagrafici e di contesto, i valori in tempo reale rilevati dai sensori, quali la temperatura interna, la temperatura esterna e il consumo istantaneo, aggiornati automaticamente, i grafici storici dei consumi e delle temperature con la possibilità di selezionare l'intervallo temporale e la grandezza di interesse, e le metriche di efficienza termica dell'edificio nel periodo selezionato, tra cui l'energia consumata, la temperatura esterna media, la dispersione termica stimata e il coefficiente di prestazione medio.

3.7 Sensori

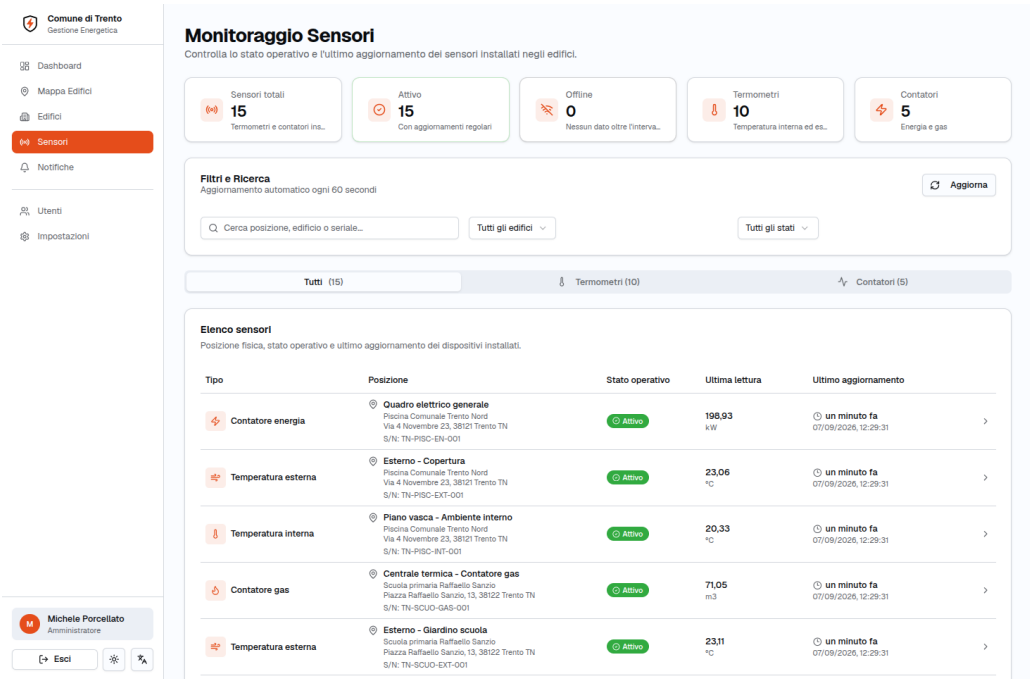


Figura 11: Sezione sensori

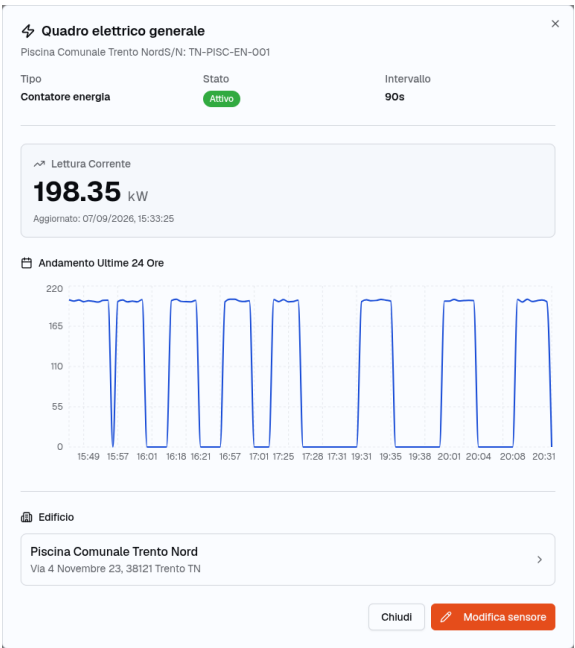


Figura 12: Dettaglio sensore

In fig. 11 è presente la sezione sensori. Essa comprende la gestione e il monitoraggio dei dispositivi installati. La pagina permette di registrare nuovi sensori, eseguire nuove ricerche e filtrarli per tipologia, oltre a vedre il loro stato operativo. Cliccando sul sensore è possibile vederne i dettagli (fig. 12); Da qui, andando su modifica sensore è possibile configurarne la tipologia, la posizione, l'intervallo di trasmissione e le soglie di allarme minima e massima.

3.8 Allarmi

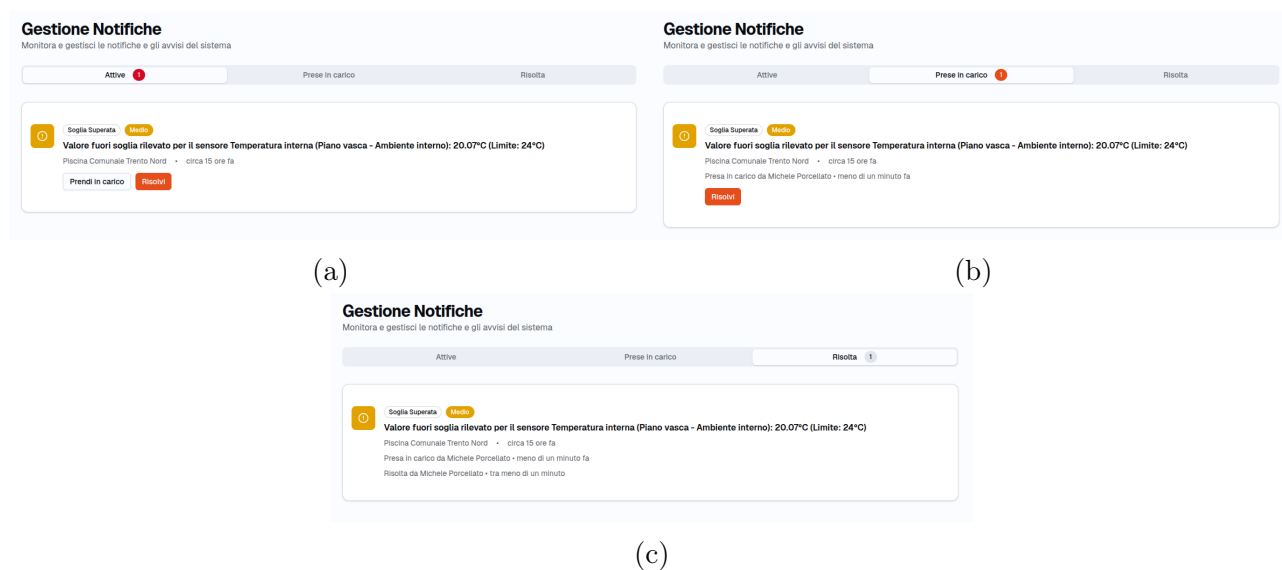


Figura 13: Pagina di gestione degli allarmi

La pagina degli allarmi (fig. 13) raggruppa le notifiche generate dal sistema. Esse sono filtrabili per stato e ordinate di default per timestamp. Per ciascun allarme è possibile consultare i dettagli, prenderlo in carico o risolverlo.

3.9 Amministrazione

Le sezioni riservate agli amministratori comprendono la gestione degli utenti e le impostazioni di sistema.

3.9.1 Utenti

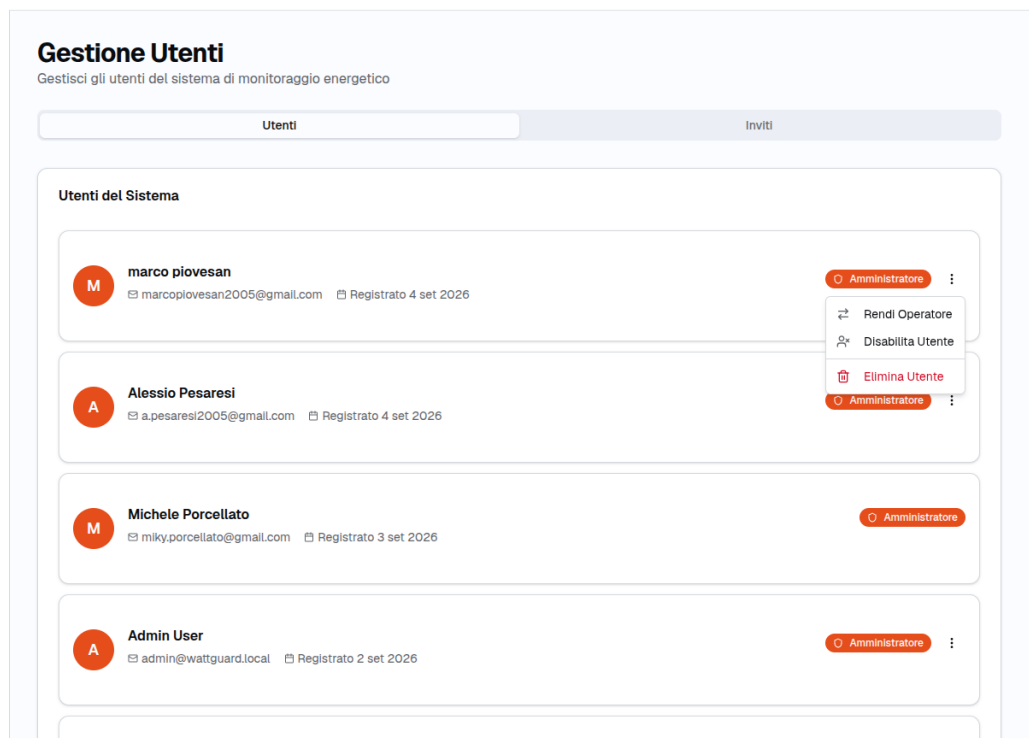


Figura 14: Sezione utenti

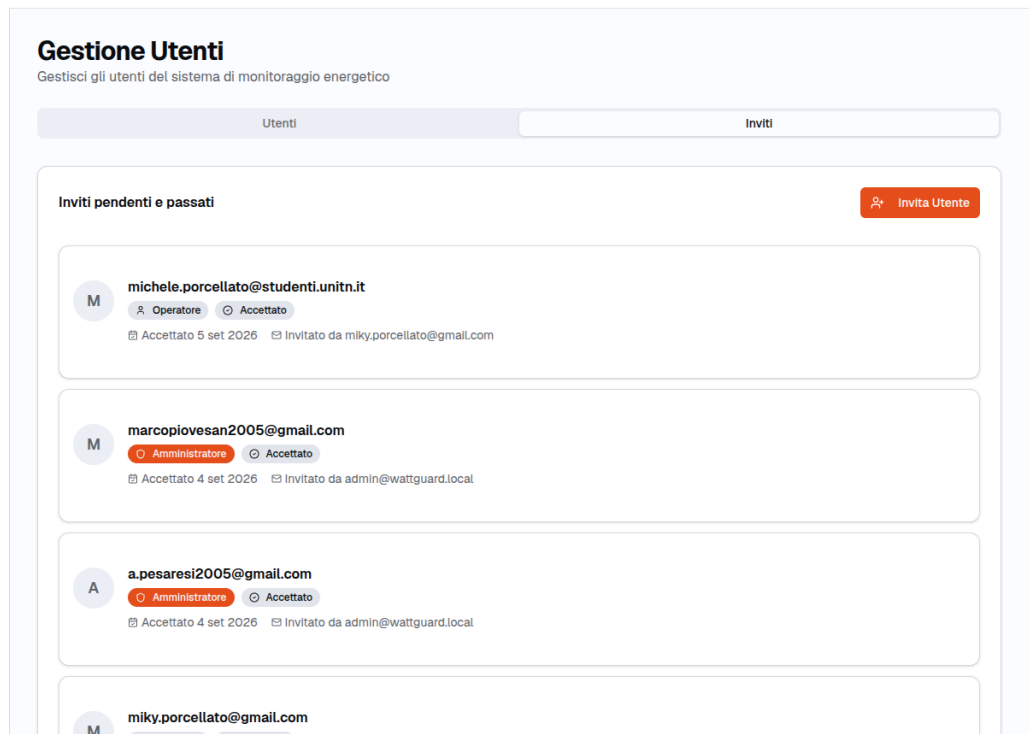


Figura 15: Sezione inviti

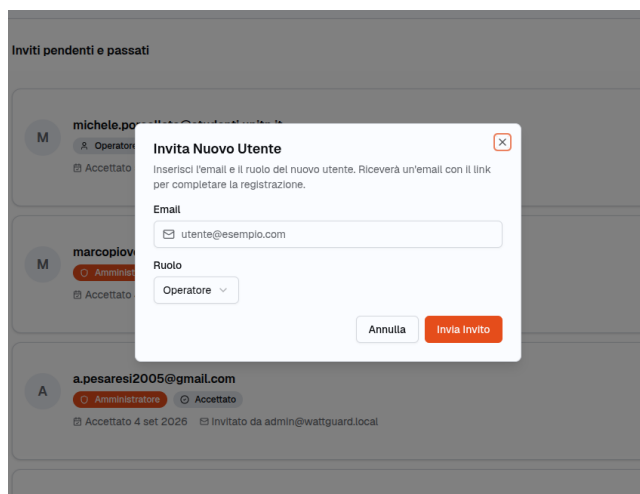


Figura 16: Dialogo invito nuovo utente

Nella sezione utenti (fig. 14) è possibile controllare gli utenti presenti, cambiare il loro ruolo, disattivarli o eliminarli completamente dal sistema.

Per aggiungere nuovi utenti, è necessario recarsi nella sezione inviti (fig. 15). Qui sono disponibili gli inviti presenti con lo stato (accettato o in attesa) e la possibilità di aggiungere nuovi utenti al sistema usando la finestra di dialogo mostrata in fig. 16.

3.9.2 Impostazioni di sistema

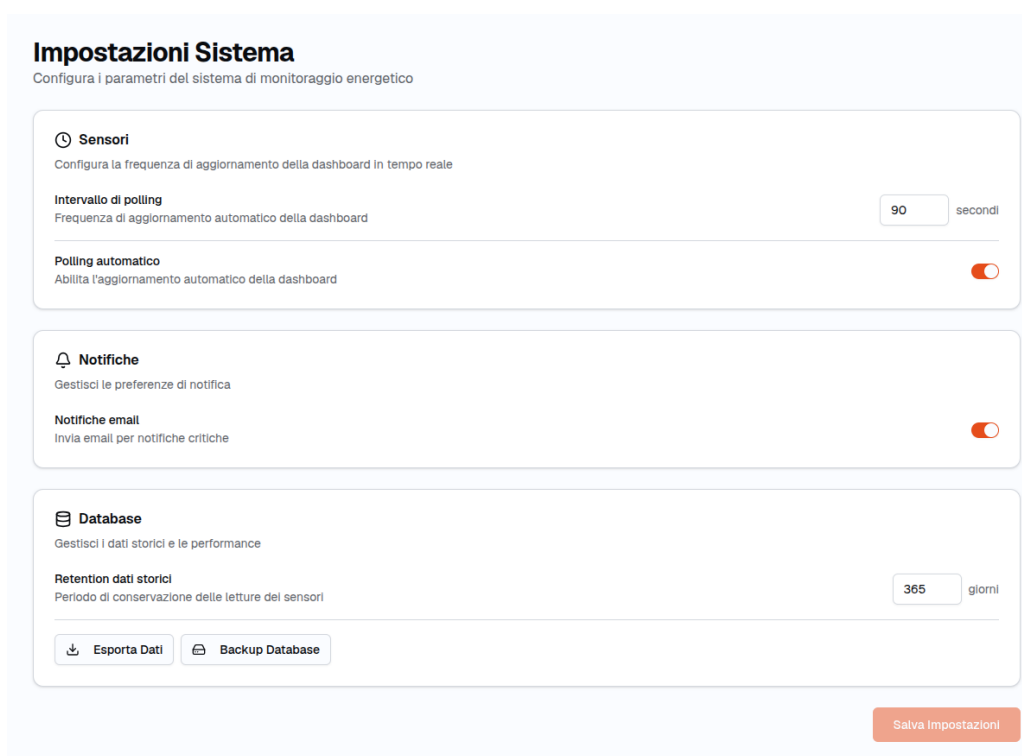


Figura 17: Impostazioni di sistema

Nella pagina di impostazioni di sistema (fig. 17) gli amministratori possono configurare l'intervallo di polling per l'aggiornamento dei valori dell'applicazione, gestire se ricevere notifiche email in caso ci siano allerte presenti e gestire i dati storici dell'applicazione (durata di conservazione, esportazione delle letture e backup del database in file JSON)

4 Deployment

Un'istanza del backend è ospitata sulla piattaforma *render.com* ed è disponibile al link wattguard.me. Sia backend che frontend vengono serviti presso questo indirizzo.

Il database MongoDB è invece ospitato sulla piattaforma Atlas.

4.1 Limitazioni del deployment

Dovute a limitazioni del piano gratuito della piattaforma Render, l'istanza dell'applicazione viene sospesa dopo 15 minuti di inattività. Appena viene ricevuta una nuova richiesta HTTP, l'istanza torna operativa in circa un minuto (Cold Start). Di conseguenza, la prima richiesta avrà una latenza maggiore, mentre le richieste successive avranno un tempo di risposta standard.

Per superare le limitazioni imposte dalla piattaforma sono state impiegate le seguenti soluzioni:

- **Migrazione da SMTP a Resend:** Il piano gratuito di Render non permette connessioni a server SMTP esterni. Per ovviare al problema si è impiegata il servizio **Resend** e la sua libreria. Questo servizio, anch'esso gratuito, permette di mandare mail tramite richieste HTTP, aggirando le restrizioni imposte.
- **Dominio certificato per le mail:** Per usufruire dei servizi di Resend è necessario possedere un dominio. Ci siamo affidati a **Namecheap** registrando il dominio `wattguard.me` e configurando i record DNS richiesti (record TXT)
- **Simulatore in-process** Dato che non disponiamo né di un broker MQTT esterno, né di ulteriori risorse di calcolo per simulare una topologia di sensori realistica, tutte le misurazioni dei sensori sono create con un simulatore interno all'applicazione. Questa scelta comporta un limite: l'eventuale sospensione dell'istanza di Render comporta l'interruzione anche del simulatore, facendo apparire i sensori offline.

4.2 Configurazione CI/CD

La configurazione CI/CD basata su GitHub Action esegue le verifiche automatiche del codice ad ogni push sui rami `main` e `develop`. Le verifiche comprendono:

- `bun run check` Controllo dei tipi statici da parte del compilatore TypeScript;
- `bun run lint` Analisi statica del codice (linting) fornita da `oxlint`;
- `bun run test` Esecuzione dei test unitari e di integrazione;
- `bun run build` Generazione dei file di bundle e degli asset finali.

Nel caso le verifiche abbiano successo e ci si trova nel ramo `main` viene anche eseguito il deploy su Render.

4.3 Credenziali e accesso docenti

Per accedere al deploy dell'applicazione è possibile utilizzare i seguenti account:

Utente amministratore

- **Email:** `admin@wattguard.local`

- Password: admin123

Utente operatore

- Email: operator@wattguard.local
- Password: operator456

In caso di problemi con il deployment contattare:

- michele.porcellato@studenti.unitn.it
- marco.piovesan-2@studenti.unitn.it
- alessio.pesaresi@studenti.unitn.it